OBSIDIAN NETWORK

Obsidian Seal Coin (OBS) · The Proof of Time Blockchain

EXCHANGE INTEGRATION & TECHNICAL DOCUMENTATION

Operational manual for centralized-exchange technical due diligence, wallet integration, deposits, withdrawals and listing.

PUBLIC TECHNICAL DOCUMENTATION

Document version	1.0.0
Date	2026-10-20
Software described	obsidian-core 2.0.4 · protocol 2.0.2 · commit c29811e (tag v2.0.4)
Networks	Mainnet chain id 7777, prefix obs1 (testnet 7778 tobs, staging 7779 sobs, devnet 7780 dobs)
Evidence	Source read at the commit above; behaviour executed on a local devnet. No live-network values are claimed.
Contact (placeholder)	emoluxlabs@gmail.com — REQUIRES OPERATOR CONFIRMATION
Website (user-supplied, NOT VERIFIED live)	https://mineobs.network/

Evidence tags used throughout: **[VERIFIED-SRC]** **[VERIFIED-DEVNET]** **[ILLUSTRATIVE]** **[NOT VERIFIED]** **[NOT IMPLEMENTED]** **[OPERATOR]**

Table of contents

1. How to read this document . . . 5

2. Executive summary . . . 5

3. Project identity and mainnet configuration summary . . . 6

3.1 Identity . . . 6

3.2 Mainnet configuration summary . . . 6

3.3 Why the live genesis id cannot be asserted from the repository . . . 8

4. Public node and network connectivity . . . 9

5. Core concepts for integrators . . . 10

5.1 Amounts . . . 10

5.2 Addresses . . . 10

5.3 The transaction lifecycle (the five states an exchange must keep apart) . . . 10

5.4 Nonce . . . 10

5.5 Gas (fee) . . . 10

5.6 Confirmations and finality . . . 11

6. Operation A — Create an address / wallet . . . 11

6.1 CLI (A3) . . . 11

6.2 Exchange recommendation: one address per customer, derived from an offline xpub . . . 12

6.3 Validation of an address (local, before storing or paying) . . . 12

6.4 Key handling . . . 12

6.5 Memo / destination tag . . . 12

6.6 Sub-procedure for deposit address issuance (checklist) . . . 12

7. Operation B — Get a balance . . . 13

7.1 Endpoint . . . 13

7.2 Semantics an exchange must understand . . . 13

7.3 Errors and retries . . . 13

7.4 Recommended balance procedure . . . 13

8. Operation C — Transfer / withdraw . . . 14

8.1 What exists . . . 14

8.2 The PAYMENT transaction (exact format) . . . 14

8.3 Fees . . . 14

8.4 Memo / tag . . . 15

8.5 Nonce handling for withdrawals . . . 15

8.6 Idempotency . . . 15

8.7 Withdrawal states and errors . . . 15

8.8 Offline signing . . . 16

8.9 Recommended withdrawal procedure . . . 16

9. Operation D — Transaction information, history and deposit detection . . . 16
9.1 Interfaces . . . 16
9.2 The deposit-detection method that works (block scanning) . . . 16
9.3 What a deposit record must contain . . . 17
9.4 Reorg handling . . . 17
9.5 Crediting policy (exchange recommendation, [OPERATOR] to ratify numbers) . . . 17
9.6 Sweeping deposits . . . 17
9.7 Retention and rate planning . . . 18

10. Workflows . . . 18
10.1 Deposit lifecycle . . . 18
10.2 Withdrawal lifecycle . . . 19
10.3 Protocol finality vs the exchange's credit policy . . . 19
10.4 Persistent records (minimum schema) . . . 20
10.5 Idempotency summary . . . 20
10.6 Timeouts, retries, node restarts . . . 20
10.7 Reconciliation . . . 20
10.8 Insufficient funds . . . 20
10.9 Reorgs . . . 20

11. API / RPC reference . . . 20
11.0 Index . . . 21
11.1 Liveness — GET /livez . . . 21
11.2 Readiness — GET /readyz . . . 22
11.3 Health and chain identity — GET /health . . . 24
11.4 Software and protocol version — GET /version . . . 24
11.5 Network definition — GET /network . . . 25
11.6 Consensus parameters — GET /params . . . 26
11.7 Genesis document and identity — GET /genesis . . . 27
11.8 Supply — GET /supply . . . 27
11.9 Chain status — GET /status . . . 28
11.10 Finalized checkpoint (operator-local) — GET /finality . . . 29
11.11 List blocks — GET /blocks?limit={1..100}&from;={height} . . . 30
11.12 Block by height or hash — GET /block/{height|hash} . . . 31
11.13 Transaction lookup — GET /tx/{txid} . . . 32
11.14 Address history (indexer) — GET /address/{address}?limit={1..100} . . . 33
11.15 Mempool snapshot — GET /mempool . . . 34
11.16 Next nonce — GET /wallet/{address}/next-nonce . . . 35
11.17 Balance — POST /wallet/balance . . . 35
11.18 Send quote (gas + sufficiency + nonce + protocol time) — POST /wallet/quote . . . 36
11.19 Gas (fee) quote — POST /tx/gas . . . 37
11.20 Encode / decode a transaction BODY — POST /tx/encode . . . 38
11.21 Simulate a signed transaction — POST /tx/simulate . . . 39
11.22 Broadcast a signed transaction — POST /tx/submit . . . 39

11.23 JSON-RPC 2.0 endpoint — POST /rpc . . . 40
11.24 Peers (operator-local) — GET /peers . . . 42
11.25 Prometheus metrics — GET /metrics . . . 42
11.26 Capabilities that do NOT exist . . . 43

12. Running your own node and exposing it safely . . . 44
12.1 Why the exchange should run its own node . . . 44
12.2 Deployment outline [VERIFIED-SRC] (commands exist; mainnet run not performed here) . . . 44
12.3 Exposure rules (RPC is unauthenticated) . . . 44

13. Security . . . 45
13.1 Key custody . . . 45
13.2 Local signing and keystore . . . 45
13.3 Transport and perimeter . . . 45
13.4 Replay and duplicate protection . . . 45
13.5 Withdrawal approvals and limits . . . 45
13.6 Backup and disaster recovery . . . 45
13.7 Monitoring and alerts . . . 45
13.8 Incident response . . . 46
13.9 Upgrades . . . 46

14. Examples . . . 46
14.1 cURL [VERIFIED-DEVNET] (replace NODE with your own node; mainnet RPC default port is 8630) . . . 46
14.2 Raw JSON: a transaction lookup and the same transaction through JSON-RPC [VERIFIED-DEVNET] . . . 46
14.3 Node.js [VERIFIED-DEVNET] . . . 46
14.4 Python [VERIFIED-DEVNET] . . . 47
14.5 CLI [VERIFIED-DEVNET] (commands run against devnet; mainnet needs --network mainnet) . . . 47
14.6 Signing test vector (devnet, public test keys — never use for value) [VERIFIED-DEVNET] . . . 47

15. Test and acceptance checklist . . . 47

16. Gap analysis . . . 49

17. Appendices . . . 55
Appendix A — Error code catalogue . . . 55
Appendix B — Where existing project documents disagree with the code . . . 55
Appendix C — Evidence index: executed devnet checks . . . 55
Appendix D — Version history . . . 58
Appendix E — Contact and distribution . . . 58

Appendix F — Test and acceptance checklist (fill-in) . . . 59
Results summary (preparation, devnet) . . . 62
Sign-off . . . 62
Notes . . . 62

1. How to read this document

This is an operational integration manual built from the repository source, not a whitepaper. Every statement carries an evidence tag. **Nothing is stated as fact unless it was read in source or executed.**

Tag	Meaning
[VERIFIED-SRC]	Read in the repository source at the commit above (file named).
[VERIFIED-DEVNET]	Executed against a local, disposable devnet node of this exact build during preparation of this document (`verification/devnet-verify.mjs`; transcripts in `verification/verification-results.json`). It proves behaviour of the software, **not** of the live mainnet.
[ILLUSTRATIVE]	Shape or placeholder values; not executed.
[NOT VERIFIED]	Cannot be established from the repository or this environment (for example anything that needs the live network, DNS or TLS).
[NOT IMPLEMENTED]	The capability does not exist in the code.
[OPERATOR]	REQUIRES OPERATOR CONFIRMATION: only the project operator can supply the value.

What was and was not possible. The preparation environment had no route to the live Obsidian mainnet, to any `obsmainnet.us.ci` or `mineobs.network` host, or to any block explorer. Therefore **no live mainnet value (height, supply, peers, endpoint reachability, TLS certificate, observed block time) is claimed anywhere in this document.** All executed tests ran on a local devnet (chain id 7780) with throw-away keys.

Environment of the executed tests: Node.js 22, `obsidian-core` 2.0.4 built from the commit above, one node, no peers, block production on, one local mining-gate issuer key committed in the devnet genesis.

2. Executive summary

What an exchange gets. A native-coin chain (no token contract) with secp256k1 keys, bech32 `obs1…` addresses, 18-decimal integer amounts, a deterministic gas formula, a single-recipient `PAYMENT` transaction with an optional memo, a plain HTTP/JSON node RPC, and offline wallet generation. The signing format is fully specified by code and was independently re-implemented in Python and checked byte-for-byte against the core (test vector in §14).

What an exchange must plan around (details in §16):

1. **Run your own node.** The node RPC has no authentication, public finality is withheld by design, and the only route that returns unmasked recipients (JSON-RPC `gettransaction`) should not be internet-facing.
2. **Use one deposit address per customer.** A memo exists but cannot be searched and is not reliably retrievable (GAP-03, GAP-16).
3. **Detect deposits by block scanning**, not by address history (REST history masks counterparties and shows at most 100 records).
4. **No push mechanism** (no WebSocket, no webhook): poll.
5. **One in-flight transaction per sending address**: withdrawals from one hot address are serialised to about one per block.
6. **Finality is visible only to an operator-local caller.** Public callers see confirmation depth only.
7. **Genesis identity, public RPC endpoints, seed peers, explorer and circulating-supply definition need operator confirmation** (GAP-08, GAP-11, GAP-12, GAP-18).

Readiness verdict (engineering view, evidence-based). Operations A (create address), B (balance), C (withdraw) and D (transaction lookup / deposit detection) are all implementable today against a self-run node, and each was exercised end-to-end on devnet. The blocking items for a **mainnet** listing are operator confirmations, not missing code: the live genesis id, a seed/peer list, the published public endpoints, the supply definition and the policy for future protocol upgrades (§16).

3. Project identity and mainnet configuration summary

3.1 Identity

Item	Value	Status
Project	Obsidian Network	user-supplied; matches repo name **[VERIFIED-SRC]**
Coin name	Obsidian Seal Coin	user-supplied; **not found in the repository** → **[NOT VERIFIED]** in repo. The repo calls the base unit a seal.
Ticker	OBS	**[VERIFIED-SRC]** (`OBS_DECIMALS`, UI and API use "OBS")
Slogan	"The Proof of Time Blockchain"	**[VERIFIED-SRC]** (`obsidian-app-web/public/screens.mjs`)
Website	https://mineobs.network/	user-supplied. Listed as a first-party trusted host in `docs/trusted-domains.md` and `obsidian-core/src/config/trusted-origins.ts` **[VERIFIED-SRC]**; DNS ownership, hosting and TLS **[NOT VERIFIED]** (the same doc says "You must own `mineobs.network`")
Web app	https://app.mineobs.network/	as above **[NOT VERIFIED]** live
Wallet	https://wallet.mineobs.network/	as above **[NOT VERIFIED]** live
Repository-documented domain family	`obsmainnet.us.ci` with `api.`, `wallet.`, `explorer.`, `mine.`, `ons.`, `node.`, `developer.` subdomains	**[VERIFIED-SRC]** `docs/trusted-domains.md`; live status **[NOT VERIFIED]**
Contact	emoluxlabs@gmail.com	placeholder supplied by the user **[OPERATOR]**
Token standard	**None. OBS is the native coin of its own chain.** There is **no ERC-20, BEP-20, TRC-20 or other contract**, no contract address and no bridge in the repository. Do not list it under a token standard unless the operator confirms a wrapped representation.	**[VERIFIED-SRC]** (no contract/bridge code; `nativeExchangeEnabled:false`)

3.2 Mainnet configuration summary

Parameter	Mainnet value	Status / source
Network name / network id	`mainnet` / `obsidian-mainnet-1`	**[VERIFIED-SRC]** `protocol/networks.ts`
Chain id	**7777** (inside every signed transaction)	**[VERIFIED-SRC]**
Address format	bech32 (BIP-173, **not** bech32m), human-readable part `obs`, 20-byte payload `RIPEMD160(SHA256(compressed pubkey))`, 43 characters on devnet `dobs…`; **mainnet addresses are 42 characters** (`obs1` + 32 data + 6 checksum), all lowercase	**[VERIFIED-SRC]** `crypto/keys.ts`, `crypto/bech32.ts`; length **[VERIFIED-DEVNET]** (devnet 43 = 4-char prefix)
Other prefixes	testnet `tobs`, staging `sobs`, devnet `dobs`	**[VERIFIED-SRC]**
Keys / signatures	secp256k1; ECDSA, RFC 6979 deterministic, 64-byte compact r‖s, **low-S enforced**	**[VERIFIED-SRC]**
HD derivation	BIP-39 (24 words, 256-bit) → BIP-32 path `m/44'/7777'/<account>'/0/<index>`; coin type 7777 is documented in code as a placeholder, not a registered SLIP-44 value	**[VERIFIED-SRC]** `crypto/mnemonic.ts`
Decimals / base unit	18 decimals; 1 OBS = 10^18 **seals**; all consensus math is integer	**[VERIFIED-SRC]** `protocol/amount.ts`
Maximum supply	21,000,000 OBS (hard cap; any issuance beyond it is rejected)	**[VERIFIED-SRC]** `MAX_SUPPLY_SEALS`
Total (issued) supply	`GET /supply` → `totalSupplyObs` (issued so far: genesis allocation + mining rewards)	**[VERIFIED-SRC]**; live value **[NOT VERIFIED]**

Parameter	Mainnet value	Status / source
Circulating supply	**No definition and no field exists** in code or docs. Do not publish a number until the operator defines it (what is excluded: Mining Pool balance, marketplace escrow, the 100,000 OBS genesis allocation holder, treasury?)	**[NOT IMPLEMENTED]** / **[OPERATOR]** (GAP-12)
Supply at height 0	0 OBS. The one-time genesis allocation (100,000 OBS) is issued only to the first protocol-valid mining claim.	**[VERIFIED-SRC]** `docs/mainnet-launch.md`, `params.ts`
Core / protocol version	core 2.0.4 · protocol 2.0.2 · wire protocol 1 · min core 2.0.2	**[VERIFIED-SRC]** `/version`
Params hash	`12211595855ba405b3d70f6161ef9371` (identical on every node of one network)	**[VERIFIED-DEVNET]** value printed by the 2.0.4 build; it is a function of the consensus constants only
Genesis id / hash (repo default, **no mining-gate key committed**)	`470a92366594851f6d22522fbc1232e8605cf9cc` / `e50e046778de21c0125872702c4691fd20af0277babca36afb668fa2dd17d012`	**[VERIFIED-SRC]** `docs/mainnet-launch.md`
Live mainnet genesis id / hash	**The identity of a running mainnet depends on the mining-gate public keys committed at genesis**; the key-less values above differ from any genesis created with a gate key (devnet proof in §3.3).	**[OPERATOR]** — GAP-11. The exchange must pin the values the operator publishes and compare them with `/genesis` on its own node.
Genesis timestamp	`1767225600` (2026-01-01T00:00:00Z) in the devnet document; mainnet's document **[OPERATOR]**	devnet **[VERIFIED-DEVNET]**; mainnet **[NOT VERIFIED]**
Consensus	Proof of Time: deterministic producer schedule over registered, mature, live nodes; no hash race, no stake, no vote. Fork choice: finalized prefix → fixed PoT weight → height → lowest block hash	**[VERIFIED-SRC]** `params.ts`, `docs/consensus-v1.9.0.md`
Block time target	5 seconds. Observed on devnet (single producer): mean 5.0 s over 8 blocks. **Mainnet observed value: [NOT VERIFIED]**	target **[VERIFIED-SRC]**; devnet **[VERIFIED-DEVNET]**
Confirmation depth hints in code	soft 12 blocks (~60 s), hard 64 blocks (~320 s), both advisory	**[VERIFIED-SRC]** `block.confirmationDepthSoft/Hard`
Finality	Producer-diversity checkpoint: block B is final when the `window(n) = clamp(3n, 32, 1024)` blocks after it were produced by at least `floor(2n/3)+1` distinct established producers; needs ≥ 2 established producers. Economic (not BFT) finality. **Visible only to an operator-local caller.**	**[VERIFIED-SRC]** `consensus/finality.ts`, `params.ts`
Maximum reorg depth	256 blocks (the node's reorganiser refuses deeper)	**[VERIFIED-SRC]** `blockchain/chain.ts`
Minimum transfer	0.000001 OBS (10^12 seals)	**[VERIFIED-SRC]** `tx.minTransfer`; rejection **[VERIFIED-DEVNET]**
Gas (fee)	`clamp(floor(amount × 2 / 10000), 0.000001 OBS, 0.01 OBS)`; must be exact; paid by the sender **on top of** the amount to the Mining Pool	**[VERIFIED-SRC]** `gas.*`; **[VERIFIED-DEVNET]**
Memo	≤ 256 UTF-8 bytes (two places, see §8.4); **no memo/tag is required**	**[VERIFIED-SRC]**
Transaction limits	16 KiB signed size; `validUntil` ≤ protocol time + 1200 s (240 blocks × 5 s)	**[VERIFIED-SRC]** `tx.maxTxBytes`, `tx.expiryBlocks`
Block limits	2 MiB, 2000 transactions	**[VERIFIED-SRC]**
Mempool	10,000 tx / 32 MiB / 64 per sender (admission additionally requires nonce = account nonce, so effectively 1 per sender)	**[VERIFIED-SRC]**, nonce rule **[VERIFIED-DEVNET]**
Default ports	RPC 8630 · P2P 8631 (testnet 18630/18631, staging 28630/28631, devnet 38630/38631)	**[VERIFIED-SRC]**
P2P magic	`OBSM`	**[VERIFIED-SRC]**

Parameter	Mainnet value	Status / source
Explorer	`explorer.obsmainnet.us.ci` is the documented name when an explorer is served on its own host	**[NOT VERIFIED]** live (GAP-18)
Native exchange / custody inside the protocol	None (`nativeExchangeEnabled:false`)	**[VERIFIED-SRC]**
Third-party security audit report	none is included in the repository	**[NOT VERIFIED]** / **[OPERATOR]** to supply if one exists

3.3 Why the live genesis id cannot be asserted from the repository

The genesis document includes `miningGatePublicKeys`. On the verification devnet, committing one gate key produced genesis id `64f5bb13a2432e8d6786ef2dd4d8d86f920f7313`, while the key-less repository default for devnet is `1eec1cf5cc764459407d7b2a8c4c3be3b23dfa24`. **[VERIFIED-DEVNET]** The same mechanism applies to mainnet (`docs/mainnet-launch.md`: "No mainnet key is committed in this repository"). A genuine mainnet therefore shows the identity of its genesis; the table value above is the key-less reference. Exchanges must treat the operator's published `genesisId`/`genesisHash`/`paramsHash` triple as authoritative and refuse any node that differs.

4. Public node and network connectivity

Item	State
Network id	`obsidian-mainnet-1` **[VERIFIED-SRC]**
Software version	core 2.0.4 / protocol 2.0.2 **[VERIFIED-SRC]**; version running on live mainnet **[NOT VERIFIED]** (2.0.2–2.0.4 nodes interoperate: same protocol identity)
Public RPC URL	**None is confirmed.** `https://api.obsmainnet.us.ci` and `https://api.mineobs.network` are named as "a node's public API" in `docs/trusted-domains.md` and the trusted-origin policy, but no repository file proves either is live, and the preparation environment could not test them. **Do not integrate against an unconfirmed endpoint.** **[NOT VERIFIED]** / **[OPERATOR]**
Public IP / hostname of a node	none in the repository; none published here **[OPERATOR]**
RPC port	8630 default; the node binds `127.0.0.1` by default (`rpcHost`), so **a configured port is not a reachable port** **[VERIFIED-SRC]** `config/mainnet.json`
TLS	The node RPC speaks plain HTTP. TLS is the reverse proxy's job (nginx/Cloudflare examples in `deployment/` and `docs/ORACLE-VPS-DEPLOYMENT.md`). Certificate status of any live host **[NOT VERIFIED]**
P2P	TCP 8631, handshake verifies networkId, chainId, exact protocol version, minimum core version, paramsHash and genesisId; peers that differ are refused **[VERIFIED-SRC]** `docs/protocol.md`
Peer discovery	seeds (`--seeds` / `OBSIDIAN_SEED_NODES`), saved peer book, peer exchange (`getaddr`). The mainnet configuration ships an empty seed list: **a seed/peer list must be obtained from the operator** **[OPERATOR]** (GAP-08)
Joining	`obsidian-core join --network mainnet --peer <http url of a node RPC> --genesis-id <40 hex>` pins the genesis id you were given, then `start --network mainnet --no-mine` runs a follower that validates but does not produce blocks **[VERIFIED-SRC]** `src/index.ts` (command present; not executed against mainnet)
Protocol / chain id / genesis hash	see §3.2
Block time target vs observed	target 5 s **[VERIFIED-SRC]**; observed on mainnet **[NOT VERIFIED]**; devnet 5.0 s **[VERIFIED-DEVNET]**
Finality	operator-local only; see §5.6
Sync status	`GET /readyz` (`sync.syncing`, `behindBy`), `GET /health` (`syncing`) **[VERIFIED-DEVNET]**
Height with timestamp	`GET /health` → `height`; `GET /status` → `lastBlockTimestamp`. Current mainnet height **[NOT VERIFIED]** — not obtainable from here
Recommended confirmations	exchange policy, not a protocol value: see §5.6 and §10.3 **[OPERATOR]** to ratify
Rate limits	240 requests/minute per client IP by default; 429 `ERR_RATE_LIMITED` **[VERIFIED-DEVNET]** (limit reached after ~200 requests including earlier calls). Behind a proxy all callers share the proxy's IP unless `OBSIDIAN_RPC_TRUST_PROXY=true`
Authentication	none at the node. No API keys, no per-client quotas (GAP-20)
IP allow-listing	not implemented in the node; do it at the firewall/proxy
Body / time limits	request body ≤ 512 KiB; headers 10 s, request 15 s, socket 20 s **[VERIFIED-SRC]**
CORS / origins	cross-origin browser calls need an allow-listed origin, otherwise 403; irrelevant to server-side exchange code
Archive / indexer	every node keeps the full chain and a persisted index (`<data-dir>/index/*.jsonl`); the index survived a restart **[VERIFIED-DEVNET]**; it rolls back on reorg **[VERIFIED-SRC]**. No separate archive/indexer service exists
WebSocket / webhook / event stream	**[NOT IMPLEMENTED]**
Should the exchange run its own node?	**Yes** (recommended; reasons in §12.1). A follower node costs no stake, bond or registration.

5. Core concepts for integrators

5.1 Amounts

Amounts are unsigned integers of seals (1 OBS = 10^18). REST/JSON-RPC responses give both a decimal OBS string and a seals string; **store and compute with the seals string as an arbitrary-precision integer**. Never use floating point. Requests accept decimal OBS strings with at most 18 decimals (`amountObs`) or seals strings (`/tx/encode` `amount`).

5.2 Addresses

`obs1` + 32 bech32 data characters + 6 checksum characters (42 chars on mainnet), lowercase. An ALL-UPPERCASE address (what QR codes carry) is valid bech32; the node lower-cases it and answers with the lowercase form, while **a mixed-case address is refused [VERIFIED-DEVNET]**. Store only the lowercase form. An address belongs to one network: `obs1…` is refused by devnet and vice versa **[VERIFIED-DEVNET]**. `.obs` names are an optional on-chain name service and must be resolved to an address before paying; a `PAYMENT` never carries a name. Exchanges should accept addresses only.

5.3 The transaction lifecycle (the five states an exchange must keep apart)

#	State	How it is observed	What it does not mean
1	**Constructed**	your code assembled the envelope	nothing is on the network
2	**Signed**	signature present; **the txid is already known** (it commits to the signature) **[VERIFIED-DEVNET]**	not broadcast
3	**Broadcast / admitted**	`POST /tx/submit` → `accepted:true`; `GET /tx/{id}` → `status:"PENDING"`	not in a block; may be dropped; **never treat as final**
4	**Included**	`GET /tx/{id}` → `confirmed:true`, `height`, `blockHash`, `confirmations ≥ 1`	can still be reorganised away (up to 256 blocks deep unless finalized)
5	**Final**	operator-local `finalizedHeight ≥ height` (own node), **or** your own depth policy	—

There is **no** `FAILED`/`DROPPED`/`EXPIRED` status: a transaction that is not in the mempool and not in a block answers 404. Decide "dropped" yourself: 404 **and** protocol time past the transaction's `validUntil` (§8.7).

5.4 Nonce

Each account has a `nonce` that starts at 0 and increases by one per included transaction. A transaction is valid only if its nonce **equals** the account's current nonce. **[VERIFIED-DEVNET]** a transaction with nonce+1 is refused while nonce is still pending in the mempool (`ERR_BAD_NONCE`), and a different transaction with the same nonce gets 409 `ERR_NONCE_PENDING`. So one sending address has **one transaction in flight at a time** (GAP-09).

5.5 Gas (fee)

Fixed by formula, no fee market, no priority fee: `gas = clamp(floor(amount × 2 / 10000), 10^12, 10^16)` seals. The transaction must carry **exactly** that value (`ERR_BAD_GAS` otherwise). The sender pays `amount + gas`; the recipient receives exactly `amount` **[VERIFIED-DEVNET]** (deposit of 12.5 OBS credited exactly 12.5; the sender lost 12.5025).

Amount	Gas
≤ 0.005 OBS	0.000001 OBS (floor)
12.5 OBS	0.0025 OBS
1,000 OBS	0.2 OBS → capped at **0.01 OBS** from 50 OBS upward

(Maximum gas is 0.01 OBS, reached when `amount × 2/10000 ≥ 0.01`, i.e. amounts ≥ 50 OBS.)

5.6 Confirmations and finality

- **Confirmations** = `head height − block height + 1`, returned by `GET /tx/{id}` and `GET /block/{h}` **[VERIFIED-DEVNET]**.
- The code marks 12 blocks (~60 s) and 64 blocks (~320 s) as advisory "buried" depths. They are **not** irreversibility.
- **Finality** is a pure function of chain state (producer-diversity checkpoint). It requires at least two established producers and a quorum of distinct producers to build on a block; with fewer, nothing is ever final and the node says `available:false` **[VERIFIED-DEVNET]** (single-node devnet: "0 established producer(s), at least 2 are required"). `docs/mainnet-launch.md` says "at least three"; the code and `docs/finality-v1.9.0.md` say two (GAP-15).
- Since 2.0.3 the finalized height is **operator-local**: `GET /finality`, `/status` finality fields and `/readyz checks.finality` are withheld from any caller that is not on the node's own loopback without proxy headers **[VERIFIED-DEVNET]**. An exchange running its own node reads them locally; an exchange using someone else's node sees only depth.
- **Recommended exchange crediting policy** (exchange-side recommendation, **[OPERATOR]** to ratify the numbers): credit small deposits at ≥ 12 confirmations, large ones at ≥ 64, and treat anything above your risk limit as pending until your own node reports `finalizedHeight ≥ block height` or ≥ 257 confirmations (deeper than the node will ever reorganise, `maxReorgDepth` 256 ≈ 21 minutes at the 5 s target).

6. Operation A — Create an address / wallet

Summary. Wallet creation is **offline and local**; there is no remote wallet-creation endpoint **[NOT IMPLEMENTED by design]**: the node never sees keys. Three supported methods, in order of preference for an exchange:

Method	Online secret needed?	Many addresses?	Status
A1. Account xpub (watch-only) derivation — offline phrase, online xpub	No	Yes (unbounded)	**[VERIFIED-DEVNET]** (`examples/node/xpub-deposit-addresses.mjs selftest`: xpub-derived addresses equal `deriveWallet()` for indices 0, 1, 7, 1000). Uses standard BIP-32 on the documented path; **not** exposed as a CLI command.
A2. `deriveWallet(phrase, account, index, undefined, hrp)` from the compiled core	Yes (phrase in the process)	Yes	**[VERIFIED-DEVNET]**
A3. CLI `obsidian-core wallet new --network mainnet`	n/a (offline)	No — always `account 0, index 0`	**[VERIFIED-DEVNET]** (ran on devnet; output keys below)

6.1 CLI (A3)

```
# [VERIFIED-DEVNET on --network devnet] --network is mandatory; there is deliberately no default
node obsidian-core/dist/index.js wallet new --network mainnet
```

Output (JSON on stdout; values below are placeholders): `network`, `chainId`, `addressHrp`, `address`, `publicKey`, `derivationPath` (`m/44'/7777'/0'/0/0`), `recoveryPhrase` (24 BIP-39 words), `warning`. The phrase is **printed once and not stored**. Run it only on an offline machine and never let stdout reach a log, a terminal recorder or CI output. Because the private key and the phrase describe the same wallet, **the phrase is the backup**.

6.2 Exchange recommendation: one address per customer, derived from an offline xpub

1. On an air-gapped machine generate a 24-word phrase (A3 or `generateRecoveryPhrase()`), record it in your key-ceremony procedure (split custody, §13).
2. Export `xpub` at `m/44'/7777'/<account>'` (`examples/node/xpub-deposit-addresses.mjs export`; prints only the xpub).
3. Import the xpub into the deposit service. It derives `address(index)` for `m/…/0/<index>`; **it holds no private key**.
4. Persist `(customerId, index, address, derivationPath, createdAt)`. Never reuse an index for another customer.
5. The sweep/hot-wallet signer (offline or HSM-backed) holds the keys; deposit addresses are swept to the hot/cold wallets (§9.6).

Mainnet addresses must carry prefix `obs`. Pass `obs` as the HRP. The same phrase derives the **same key on every network** (only the prefix differs): use a separate phrase per network and never reuse a test-network phrase on mainnet **[VERIFIED-SRC]** (the CLI says so).

6.3 Validation of an address (local, before storing or paying)

An address is valid iff: lowercase; prefix `obs` + separator `1`; characters from the bech32 alphabet; bech32 checksum (constant 1, **not** bech32m); decoded payload exactly 20 bytes. Reference implementations: Node `isValidAddress(address, 'obs')` in the core, Python `obs_tx.is_valid_address` (`examples/python/obs_tx.py`, checked by `--selftest`). Server-side cross-check: `POST /wallet/balance` → 400 `ERR_BAD_ADDRESS` **[VERIFIED-DEVNET]** for `obs1invalid`, empty input, a bad checksum and a different network's prefix.

```
# [VERIFIED-DEVNET] — invalid address on the balance route
POST /wallet/balance  {"address":"obs1invalid"}
→ 400 {"error":"invalid wallet address","code":"ERR_BAD_ADDRESS"}
```

6.4 Key handling

- Never derive keys from customer data (the core states it never does). Generate from CSPRNG only.
- The core's own keystore is **node identity** material (`keygen`, `node-key.json`: AES-256-GCM, scrypt N=2^17). `obsidian-core keygen` **creates a node identity, not an exchange wallet. Do not use it for funds.**
- The project's browser wallets store a PBKDF2-600k / AES-256-GCM vault in the browser; that is for end users, not for an exchange hot wallet.
- Secrets must never appear in logs, tickets, chat or CI. The harness and examples in this package never print a key.

6.5 Memo / destination tag

Not required, not used by the protocol for routing. One deposit address per customer is the supported design (§9).

6.6 Sub-procedure for deposit address issuance (checklist)

1. Derive `address(index)` from the xpub. 2. Validate locally. 3. Persist atomically with the customer record. 4. Show the address as lowercase text **and** as a QR (QR may be upper-case; lower-case it on input). 5. Display the network (`mainnet`, chain 7777), the prefix `obs1`, the minimum deposit and the confirmation requirement. 6. Warn that other networks' addresses (`tobs1…`, `dobs1…`) and `.obs` names are not accepted.

7. Operation B — Get a balance

7.1 Endpoint

`POST /wallet/balance` on the node RPC. Headers: `Content-Type: application/json`. Auth: none. Body `{"address": "<obs1…>"}`.

```
# [VERIFIED-DEVNET] (devnet host/port and dobs address; on mainnet use YOUR node and an obs1 address)
curl -s -X POST http://127.0.0.1:38630/wallet/balance \
  -H 'content-type: application/json' \
  -d '{"address":"dobs1…"}'
```

Real response (devnet transcript, trimmed):

```
{
  "address": "dobs1gkkd2ycemx7rthc8qvx6j5txs6wx0qkthakv48",
  "balanceObs": "12.500000000000000000",
  "balanceSeals": "1250000000000000000",
  "spendableSeals": "1250000000000000000",
  "lockedSeals": "0",
  "totalSeals": "1250000000000000000",
  "nonce": 0, "txCount": 0,
  "receivedObs": "12.500000000000000000", "sentObs": "0.000000000000000000",
  "createdAtHeight": 6, "atHeight": 6, "custodial": false
}
```

7.2 Semantics an exchange must understand

Field	Meaning
`balanceSeals` = `spendableSeals`	amount spendable **now** at the head block (0 confirmations)
`lockedSeals`	OBS locked in ONS-marketplace escrow by pending offers (never part of a normal deposit address)
`totalSeals`	spendable + locked
`nonce`	next nonce this address must use
`atHeight`	head height the answer was read at

- **Balance ≠ confirmed balance.** It includes the newest block. There is **no pending/unconfirmed balance field and no "confirmed-only" variant [NOT IMPLEMENTED]**. Your credited customer balance must come from your own ledger of deposits that reached your confirmation policy; use `/wallet/balance` for **reconciliation and sweeping**, not for crediting.
- An unknown address returns zeros and `createdAtHeight:null` (not an error) **[VERIFIED-SRC]**.
- JSON-RPC alternative: `getbalance {address}` → `{address, balanceObs, nonce}` **[VERIFIED-DEVNET]**.
- ALL-UPPERCASE input is accepted and returned in lowercase; mixed case → 400 **[VERIFIED-DEVNET]**.

7.3 Errors and retries

400 `ERR_BAD_ADDRESS` / `ERR_MALFORMED` (do not retry), 429 `ERR_RATE_LIMITED` (back off; no `Retry-After` header is sent, use exponential backoff from 1 s), 5xx or timeout (retry the same read; reads are idempotent), connection refused (fail over to a second node you operate).

7.4 Recommended balance procedure

1. Check `/readyz` is 200 and `/health` identity matches your pinned triple. 2. Read balances from your own node.
3. Compare `Σ(credited deposits) − Σ(swept) − Σ(withdrawn incl. gas)` with the on-chain balance per address; a difference is an alert, not an adjustment. 4. Run reconciliation per block batch and daily in full (§10.7).

8. Operation C — Transfer / withdraw

8.1 What exists

Step	Interface	Status
Fetch nonce, balance, gas, protocol time	`POST /wallet/quote {address, amountObs}`	**[VERIFIED-DEVNET]**
Gas only	`POST /tx/gas {amountObs}`	**[VERIFIED-DEVNET]**
Encode PAYMENT body (cross-check)	`POST /tx/encode`	**[VERIFIED-DEVNET]**
Build + sign	**local code only** (Node: compiled core; Python: `obs_tx.py`) — no CLI command and no server endpoint	**[VERIFIED-DEVNET]**; CLI / server signing **[NOT IMPLEMENTED]**
Dry run	`POST /tx/simulate {tx}`	**[VERIFIED-DEVNET]**
Broadcast	`POST /tx/submit {tx}`	**[VERIFIED-DEVNET]**
Inclusion / confirmations	`GET /tx/{txid}`	**[VERIFIED-DEVNET]**

8.2 The PAYMENT transaction (exact format)

All integers big-endian; strings are UTF-8 with a `u32` byte-length prefix; byte strings likewise (`obsidian-core/src/protocol/encoding.ts`). `docs/transaction-format.md` summarises this but contains inaccuracies versus the code (it says bech32m addresses, `u64` nonce, `u8` type; the code uses **bech32**, `u32` **nonce**, `u16` **type**): **trust this section and the test vector** (GAP-07).

```
body (PAYMENT):     string to            (canonical lowercase address of THIS network)
                    u128   amount        (seals)
                    string memo          ("" when absent; ≤ 256 UTF-8 bytes)

signing preimage:   string protocolVersion   ("2.0.2")
                    u32    chainId           (7777 on mainnet)
                    string sender
                    u32    nonce
                    u16    type              (PAYMENT = 1)
                    u128   gas               (exactly expectedGas(amount))
                    u64    validUntil        (protocol-time seconds)
                    string envelopeMemo      ("" normally)
                    bytes  body

digest      = SHA256( u32(len(D1)) ‖ D1 ‖ preimage )           D1 = "OBSIDIAN:TX_SIGNING:v1"
signature   = ECDSA-secp256k1(digest, privateKey), RFC 6979, low-S, 64 bytes r‖s
txid        = SHA256( u32(len(D2)) ‖ D2 ‖ preimage ‖ pubkey33 ‖ sig64 )     D2 = "OBSIDIAN:TX:v1"   (64 hex)
signed tx   = string protocolVersion ‖ u32 chainId ‖ string sender ‖ u32 nonce ‖ u16 type ‖ u128 gas ‖ u64 validUntil
              ‖ string envelopeMemo ‖ bytes body ‖ string(pubkey hex) ‖ string(signature hex)
broadcast   = lower-case hex of the signed-tx bytes
```

Replay protection. The chain id and the nonce are inside the signed preimage, so a mainnet transaction cannot be replayed on another network or at another nonce position **[VERIFIED-SRC]**. The protocol version is signed too: use the version string the node reports (`/health.protocolVersion`, currently `2.0.2`). The txid commits to the signature, and the txid is known **before** broadcast **[VERIFIED-DEVNET]**.

8.3 Fees

See §5.5. Compute gas with integer arithmetic, include it in the signed transaction, and budget `amount + gas` from the hot wallet. To empty an address ("sweep"), solve `amount + gas(amount) ≤ balance`: reduce `amount` from `balance − gas(balance)` until it holds (`examples` and the harness do this; the remainder, a few seals up to a rounding unit, stays on the address) **[VERIFIED-DEVNET]** (F4: a 12.5 OBS deposit left 5·10^11 seals after the sweep).

8.4 Memo / tag

A PAYMENT has **two** memo fields, both signed and both ≤ 256 UTF-8 bytes: the **body memo** (what the project's wallets write) and the **envelope memo**.

Memo	Visible in `GET /tx/{id}`?	Visible elsewhere
body memo	**No [VERIFIED-DEVNET]**	`PAYMENT` event in `GET /block/{height}` `events[].data.memo`, **only for recent blocks** (the node searches its newest 200 events, GAP-03)
envelope memo	**Yes** (`memo`) **[VERIFIED-DEVNET]**	the index record

No memo/tag is required for a deposit, and an exchange should not depend on one for crediting (GAP-16). For **withdrawals** you may put your internal withdrawal id in the body memo for your own audit trail.

8.5 Nonce handling for withdrawals

- Read `nextNonce` from `/wallet/quote` or `/wallet/{addr}/next-nonce` immediately before signing.
- Only one transaction per sending address may be in the mempool. Do not pre-sign a chain of nonces (nonce+1 is refused until nonce is mined **[VERIFIED-DEVNET]**).
- **Never re-sign a withdrawal with a new nonce while the first signed transaction might still be mined**: that is how double payment happens. Rebroadcast the **same signed bytes**.
- Parallelism = more sending addresses (a pool of hot addresses), one in-flight tx each (GAP-09).

8.6 Idempotency

Situation	Behaviour	Status
Same signed bytes re-submitted while pending	HTTP 200 `{accepted:true, duplicate:true}`	**[VERIFIED-DEVNET]**
Same signed bytes re-submitted after inclusion	400 `ERR_REJECTED ERR_BAD_NONCE: expected nonce 1, received 0` — it is **not** applied twice	**[VERIFIED-DEVNET]**
Different tx, same sender+nonce, first pending	409 `ERR_NONCE_PENDING` with `pendingTxId`	**[VERIFIED-DEVNET]**

Because the txid is deterministic and known at signing time, make **txid** your idempotency key: persist `(withdrawalId, txid, signedHex, nonce, validUntil)` before the first broadcast.

8.7 Withdrawal states and errors

Exchange state	Meaning	Next
`REQUESTED` → `APPROVED`	business rules and approvals passed	build
`SIGNED`	signed bytes and txid persisted	broadcast
`BROADCAST`	`accepted:true` (admission)	poll
`INCLUDED(n)`	`/tx` `confirmed`, `confirmations = n`	wait for policy
`FINAL`	policy satisfied (or own-node `finalizedHeight ≥ height`)	notify customer
`STUCK`	404 after broadcast and `now > validUntil`	re-evaluate with a fresh nonce **only after** proving the old tx cannot be mined (past `validUntil`)

Error (REST `code` / text)	Cause	Action
`ERR_BAD_ADDRESS`	invalid recipient or another network's prefix	reject the withdrawal request
`ERR_REJECTED ERR_AMOUNT_ZERO`	below 10^12 seals	reject
`ERR_REJECTED ERR_BAD_GAS`	gas ≠ formula	fix the code; never retry blindly
`ERR_REJECTED ERR_BAD_NONCE`	nonce ≠ account nonce	re-read nonce; check whether your tx was already mined before re-signing
`ERR_REJECTED ERR_INSUFFICIENT_FUNDS`	balance < amount + gas	top up hot wallet; do not debit the customer twice

Error (REST `code` / text)	Cause	Action
`ERR_REJECTED ERR_EXPIRED`	`validUntil` already passed	re-sign **only if** the previous signed tx cannot still be mined
`ERR_WRONG_CHAIN_ID`	tx chainId ≠ node chainId	critical misconfiguration: stop
`ERR_NONCE_PENDING` (409)	a different tx holds this nonce	wait; poll `pendingTxId`
`ERR_RATE_LIMITED` (429)	per-IP limit or mempool refused	back off
`ERR_FORBIDDEN` (403)	node has submission disabled	use your own node
timeout / connection reset	unknown outcome	**re-submit the same signed bytes** (idempotent), then poll `/tx/{txid}`

`validUntil` must be ≥ the head's protocol time and ≤ protocol time + 1200 s; the project's wallets use `lastBlockTimestamp + 600`. Use `protocolTime` from `/wallet/quote` (never your server clock) **[VERIFIED-DEVNET]**: a `validUntil` in the past is refused with `ERR_EXPIRED`.

8.8 Offline signing

Signing needs only: sender key, `chainId`, protocol version, nonce, `validUntil`, amount and recipient. Fetch `nextNonce`/`protocolTime` online, transfer them to the offline signer, sign, carry the signed hex back online and broadcast. Keep `validUntil` short enough that a carried transaction expires if the courier is delayed (≤ 20 min is the protocol maximum anyway).

8.9 Recommended withdrawal procedure

1. Validate address locally; check minimum; compute `gas`. 2. Check hot-wallet `sufficient` via `/wallet/quote`. 3. Apply limits and approvals (§13). 4. Reserve the customer's funds in your ledger (customer debit `amount` + your fee policy; **the network gas is paid from the hot wallet on top of** `amount`). 5. Sign (HSM/offline). 6. **Persist txid + signed bytes**. 7. `simulate` (optional) then `submit`. 8. Poll `/tx/{txid}`; record height/blockHash. 9. Wait for policy. 10. Mark final; reconcile.

9. Operation D — Transaction information, history and deposit detection

9.1 Interfaces

Need	Interface	Caveat
Status by txid	`GET /tx/{txid}`	masked sender/recipient; amount exact
Unmasked tx data by txid	JSON-RPC `gettransaction {txId}`	**only included txs**; no confirmations; no memo; no pending state
Block list / cursor	`GET /blocks?from=H&limit=100`	hash, prevHash, txCount
Block contents	`GET /block/{height	hash}`
Address history	`GET /address/{addr}?limit≤100`	masked counterparties; newest 100; indexer, not authoritative
Pending txs	`GET /mempool`	first 100, masked sender
Balance	`POST /wallet/balance`	head state

9.2 The deposit-detection method that works (block scanning)

Because REST masks counterparties and `/block` omits the recipient, deposit detection is a **two-step scan against your own node**:

```
cursor = last fully processed height (persisted)            # start at (head - reorgWindow) on restart
loop:
  head   = GET /health .height
  for h in cursor+1 .. head:
      b = GET /block/h                                       # lists tx ids and types
      for t in b.transactions where t.type == "PAYMENT":
          r = POST /rpc {"method":"gettransaction","params":{"txId": t.id}}   # UNMASKED recipient, amount, sender
          if r.recipient in OUR_ADDRESS_SET: record deposit(txid, h, blockHash, index, to, amount, confirmations =
              ↪ head-h+1)
  verify chain continuity: blocks[h].prevHash == stored blockHash[h-1]   (else reorg → §9.4)
  sleep ~2 s
```

[VERIFIED-DEVNET]: each ingredient (block listing with the full sender and the tx type, `gettransaction` returning the full recipient and the exact amount, `/blocks?from=` with `prevHash`, confirmations that grow) was exercised; the loop itself is an ILLUSTRATIVE composition. Cost is one extra call per PAYMENT; acceptable at this chain's throughput (≤ 2000 tx/block). **Alternative for small deployments:** poll `GET /address/{depositAddress}` (history by full address; the response masks counterparties but the match is exact because you query by the full address), reading `txId`, `amount`, `height`, `blockHash`. It is capped at the newest 100 records and is an index, so use it only as a cross-check.

9.3 What a deposit record must contain

`txid`, `height`, `blockHash`, `index` in block, `recipient` (full), `amount` seals (string), `sender` (full, from `gettransaction`), `firstSeenAt`, `status` (`PENDING` / `INCLUDED` / `CREDITED` / `REORGED`), `confirmations at last check`, credited-at. Unique key: **(txid)** (a txid appears once in a canonical chain).

9.4 Reorg handling

- Keep `(height → blockHash)` for at least the last 300 blocks. At every poll compare `GET /block/{h}.hash` for your last 300 heights, or compare `prevHash` links from `/blocks?from=`.
- On mismatch at height `k`: mark every deposit with `height ≥ k` as `REORGED/PENDING`, **do not delete it**, rescan from `k`. A transaction may be re-included at a different height (same txid) or disappear.
- If you had already credited a reorged deposit: it is a **negative-balance risk** — hence a conservative policy: credit only past your depth threshold (§5.6).
- The node's index rolls back automatically on reorg (`Indexer.rollbackTo`) **[VERIFIED-SRC]**; a rolled-back transaction answers 404 until re-included.
- A reorg test is **not** part of the devnet run (single producer): reorg handling is **[NOT TESTED]** end-to-end in this package (see checklist).

9.5 Crediting policy (exchange recommendation, [OPERATOR] to ratify numbers)

Deposit size (exchange-defined)	Credit when
below your small-deposit limit	≥ 12 confirmations (~60 s)
below your large-deposit limit	≥ 64 confirmations (~5 min)
above	≥ 257 confirmations (beyond the maximum reorg depth) **or** your own node reports `finalizedHeight ≥ height`

Protocol finality is the chain's property; **your credit policy is yours**: they are different things and should be documented separately in the listing file.

9.6 Sweeping deposits

After the credit threshold: sign `PAYMENT deposit-address → hot/cold wallet` with `amount = balance − gas` (fee-inclusive solution of §8.3). The deposit address must hold at least `0.000001 OBS` minimum transfer plus gas. Each deposit address is its own sender, so sweeps run in parallel across addresses. **Dust:** an address holding less than `minTransfer + minGas` (0.000002 OBS) cannot be swept.

9.7 Retention and rate planning

At the 5 s target one poll per 2–5 s is enough. A full rescan of N blocks costs N `/block` calls (≤ 100 per `/blocks` page for the cursor): keep scans under the 240 requests/minute per-IP limit **or raise** `OBSIDIAN_RPC_RATE_LIMIT_PER_MINUTE` **on your private node** (§12).

10. Workflows

10.1 Deposit lifecycle

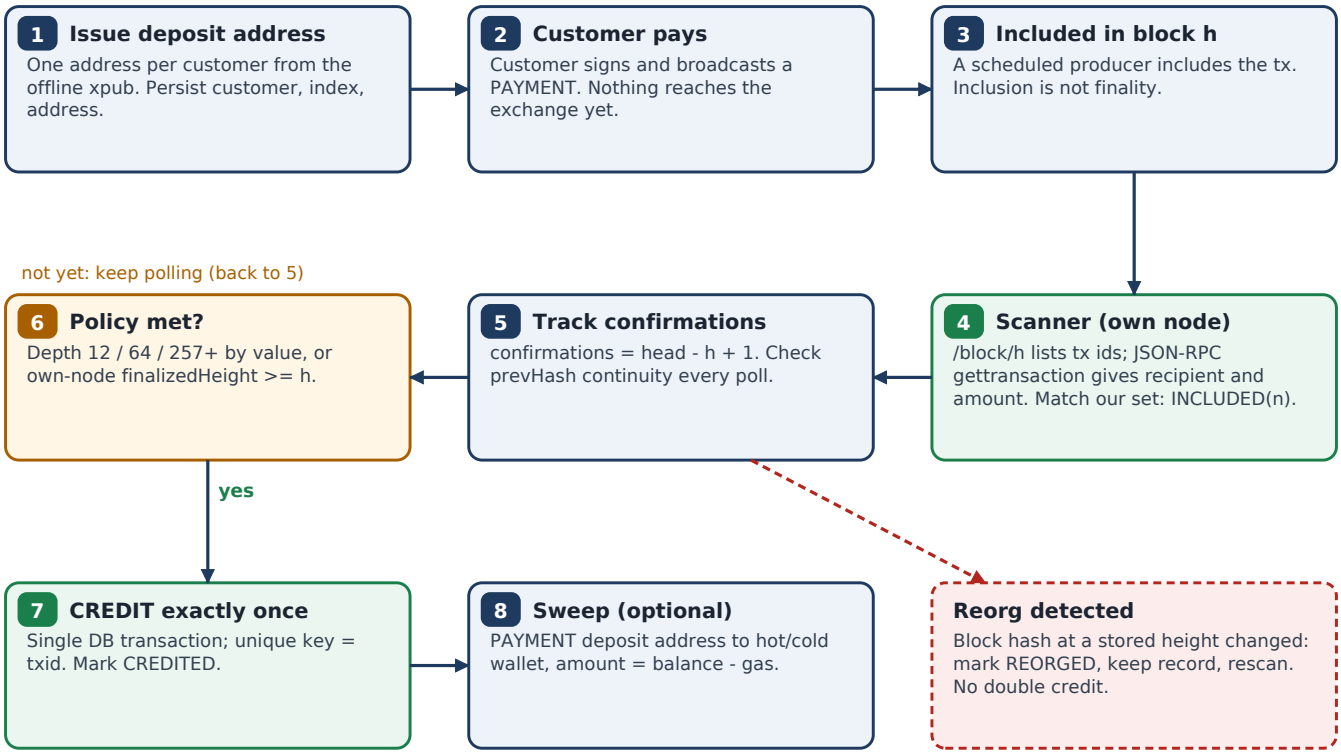


Figure 1 — Deposit lifecycle (numbers match the steps below). Green = exchange-side crediting; red dashed = failure path.

1. **Issue** a deposit address for the customer (§6.2) and persist it.
2. The customer sends OBS from any wallet. The transfer is **constructed, signed and broadcast by the customer**: nothing reaches you yet.
3. Your **scanner** (§9.2) reads new blocks from **your own node** and finds a `PAYMENT` whose recipient is one of your addresses. Record it as `INCLUDED(n)` with txid, height, blockHash.
4. Every poll: update `confirmations = head − height + 1`; check chain continuity (§9.4).
5. When the crediting policy (§9.5) is satisfied, **credit the customer exactly once** (unique key `txid`), inside one database transaction that also marks the deposit `CREDITED`.
6. Optionally **sweep** to the hot/cold wallet (§9.6) and record the sweep txid.
7. If a reorg removes the block, mark `REORGED` and re-evaluate (§9.4).

10.2 Withdrawal lifecycle

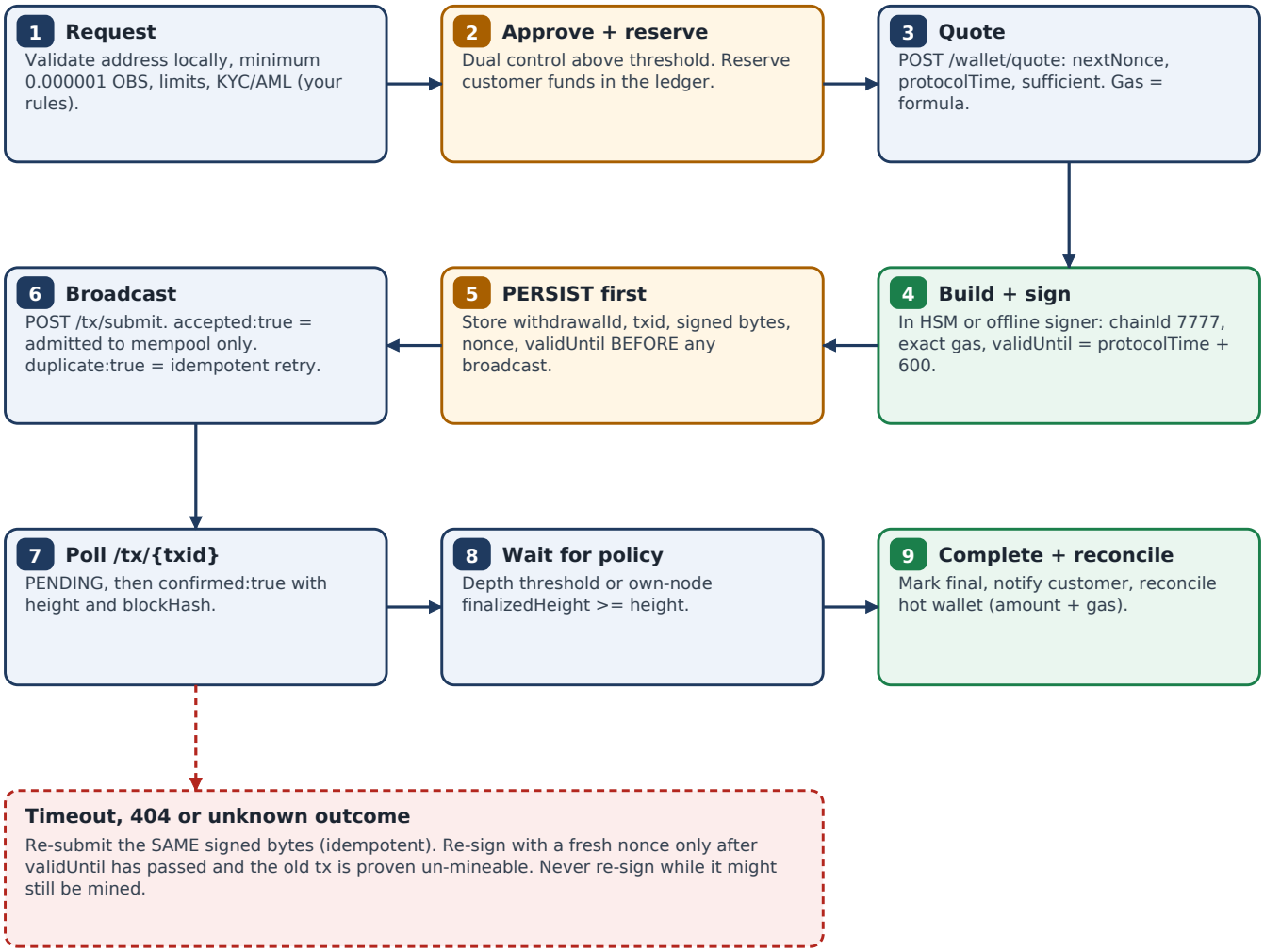


Figure 2 — Withdrawal lifecycle (numbers match the steps below). Broadcast is not inclusion and inclusion is not finality.

1. Customer request → validate address (local), minimum, limits, KYC/AML (your rules).
2. Approval workflow (dual control above a threshold, §13.5). Reserve the funds.
3. Fetch `nextNonce`, `protocolTime`, `sufficient` (`/wallet/quote`). Compute gas (`/tx/gas` as cross-check).
4. Build and **sign** in the signer (HSM/offline). Obtain `txid`.
5. **Persist** `(withdrawalId, txid, signedHex, nonce, validUntil, from, to, amount, gas)` **before** broadcasting.
6. `POST /tx/submit`. Treat `accepted:true` (or `duplicate:true`) as admitted, never as paid.
7. Poll `GET /tx/{txid}` until `confirmed:true`; persist `height`, `blockHash`.
8. Wait for the policy depth (or own-node finality). Mark the withdrawal complete and notify the customer.
9. On timeout, rebroadcast the **same bytes**; only after `validUntil` has passed with the tx still unknown (404) may you consider re-signing, and then with a fresh nonce read from the node.

10.3 Protocol finality vs the exchange's credit policy

	Protocol	Exchange
Inclusion	a block contains the tx	not enough to credit
Depth	advisory 12 / 64 blocks	thresholds you define by value and risk
Finality	`finalizedHeight` (operator-local; needs ≥ 2 established producers)	optional stricter gate; **only observable on your own node**
Reorg bound	256 blocks	alert on any reorg ≥ 1 block touching a credited deposit

10.4 Persistent records (minimum schema)

`deposits(txid PK, height, blockHash, idx, to, from, amountSeals, status, confirmations, creditedAt, sweptTxid)`; `withdrawals(id PK, txid UNIQUE, nonce, signedHex, validUntil, from, to, amountSeals, gasSeals, status, height, blockHash, createdAt, approvals[])`; `addresses(address PK, customerId, derivationPath, createdAt)`; `scan_cursor(height, blockHash)`; `block_hashes(height PK, hash)` (rolling ≥ 300); `ledger(entryId, customerId, delta, ref txid|withdrawalId)` append-only.

10.5 Idempotency summary

Deposits: unique `txid`. Withdrawals: unique `withdrawalId` **and** unique `txid`; resubmission of the same signed bytes is safe on-chain (§8.6). Never create a second signed transaction for the same withdrawal unless the first is proven un-mineable.

10.6 Timeouts, retries, node restarts

- Reads: retry with exponential backoff (1 s → 30 s); fail over to a second node you run; never mix answers from nodes with different `genesisId`.
- `submit` timeout: unknown outcome → resubmit the same bytes, then poll the txid.
- Node restart **[VERIFIED-DEVNET]**: after a restart the chain height continued, a confirmed tx was still found by id and balances were unchanged (the persisted index and state reload). During start-up `/readyz` is 503 until synced; pause the scanner and withdrawals meanwhile.
- Scanner restart: resume from `cursor − 300` and rely on the unique key to deduplicate.
- Node behind (`syncing:true` or `behindBy > 1`): do not credit and do not sign withdrawals.

10.7 Reconciliation

Per run: (a) for every deposit address, `balance(on-chain) == Σ received (confirmed) − Σ swept − Σ gas` within one rounding unit; (b) hot wallet balance == opening + Σ sweeps in − Σ (withdrawals + gas) out; (c) no `withdrawals` row stuck in `SIGNED/BROADCAST` beyond `validUntil + margin`; (d) the supply invariant `GET /health.supplyOk == true`; (e) your node's `genesisId`, `chainId` equal your pins. Any difference pages a human; nothing is auto-corrected.

10.8 Insufficient funds

On `sufficient:false` or `ERR_INSUFFICIENT_FUNDS`: do **not** re-debit; return the withdrawal to `QUEUED_FUNDING`, alert treasury, retry after top-up. Remember gas is on top of the amount.

10.9 Reorgs

See §9.4. Withdrawals: after inclusion at height `h`, keep watching `block(h).hash`; if it changes, the transaction may be re-included or vanish: keep the signed bytes and treat the withdrawal as `BROADCAST` again; rebroadcast the same bytes.

11. API / RPC reference

Transport for everything below: **HTTP/1.1 + JSON** on the node RPC port (mainnet 8630). `$NODE` is the base URL of **your own node** (for example `http://127.0.0.1:8630`); no public endpoint is confirmed (§4). Responses are `application/json` except `/metrics`. Request bodies are JSON objects (≤ 512 KiB). Errors always have the shape `{"error": string, "code": string}` except JSON-RPC (HTTP 200 + `error` object). Success examples are **real transcripts from the devnet run** (addresses carry the devnet prefix `dobs`; on mainnet they are `obs1…`), trimmed for length; error examples are tagged.

Common to every method: auth, rate limits and timeouts are stated per method; shared error statuses are 429 `ERR_RATE_LIMITED`, 404 `ERR_NOT_FOUND`, 405 `ERR_METHOD_NOT_ALLOWED`, 413 `ERR_BODY_TOO_LARGE`, 500 `ERR_INTERNAL`.

11.0 Index

#	Method	Path	Purpose (short)
11.1	GET	`/livez`	Liveness
11.2	GET	`/readyz`	Readiness
11.3	GET	`/health`	Health and chain identity
11.4	GET	`/version`	Software and protocol version
11.5	GET	`/network`	Network definition
11.6	GET	`/params`	Consensus parameters
11.7	GET	`/genesis`	Genesis document and identity
11.8	GET	`/supply`	Supply
11.9	GET	`/status`	Chain status
11.10	GET	`/finality`	Finalized checkpoint (operator-local)
11.11	GET	`/blocks?limit={1..100}&from={height}`	List blocks
11.12	GET	`/block/{height|hash}`	Block by height or hash
11.13	GET	`/tx/{txid}`	Transaction lookup
11.14	GET	`/address/{address}?limit={1..100}`	Address history (indexer)
11.15	GET	`/mempool`	Mempool snapshot
11.16	GET	`/wallet/{address}/next-nonce`	Next nonce
11.17	POST	`/wallet/balance`	Balance
11.18	POST	`/wallet/quote`	Send quote (gas + sufficiency + nonce + protocol time)
11.19	POST	`/tx/gas`	Gas (fee) quote
11.20	POST	`/tx/encode`	Encode / decode a transaction BODY
11.21	POST	`/tx/simulate`	Simulate a signed transaction
11.22	POST	`/tx/submit`	Broadcast a signed transaction
11.23	POST	`/rpc`	JSON-RPC 2.0 endpoint
11.24	GET	`/peers`	Peers (operator-local)
11.25	GET	`/metrics`	Prometheus metrics

11.1 Liveness — `GET /livez`

Field	Value
Name	`livez` — Liveness
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /livez`
Purpose	Process is up and answering. Does no chain work; never fails for chain reasons. Use for process supervision only.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe to expose; reveals only uptime.
Status / error codes	200 OK · plus the common statuses above

Field	Value
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	obsidian-core/src/rpc/server.ts `livez`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/livez"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "status": "alive",
  "uptimeSeconds": 20,
  "timestamp": 1792524563
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.2 Readiness — `GET /readyz`

Field	Value
Name	`readyz` — Readiness
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /readyz`
Purpose	Can this node be trusted to answer for the chain right now? 200 = ready (or ready-degraded), 503 = not ready (syncing, no tip, or P2P listener failed). Gate your integration on this, not on /livez.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe. Finality fields appear only for an operator-local caller (since 2.0.3); a proxied caller gets the same document without `checks.finality`.
Status / error codes	200 OK · 503 `(body status` not-ready`)` (syncing, behind peers by more than 1 block, no tip, or P2P not listening) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`readyz`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/readyz"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "status": "ready-degraded",
  "checks": {
    "chain": {
      "ok": true,
      "height": 4,
      "headHash": "fcf8aaa4012c8c4876403ffcef12d0850b12a836de42dc16f1e76ea6f6242381"
    },
    "sync": {
      "ok": true,
      "syncing": false,
      "behindBy": 0
    },
    "p2p": {
      "ok": true,
      "enabled": true,
      "listening": true,
      "connectedPeers": 0
    },
    "rpc": {
      "ok": true
    },
    "identity": {
      "ok": true,
      "genesisId": "64f5bb13a2432e8d6786ef2dd4d8d86f920f7313",
      "networkId": "obsidian-devnet-1"
    },
    "finality": {
      "ok": true,
      "finalizedHeight": 0,
      "lag": 4,
      "maxLag": 64
    },
    "gateAuthority": {
      "ok": false,
      "open": true,
      "activeKeys": 1,
      "quorum": 1,
      "epoch": 0,
      "warnings": [
        "gate-single-key"
      ]
    },
    "lastBlockAgeSeconds": 4
  },
  "degraded": [

    "no-peers"
  ],
  "advisories": [
    "gate-single-key"
  ],
  "network": "devnet",
  "genesisId": "64f5bb13a2432e8d6786ef2dd4d8d86f920f7313",
  "timestamp": 1792524563
}
```


Example error response [ILLUSTRATIVE] (HTTP 503)


```
{
  "status": "not-ready",
  "checks": {
    "sync": {
      "ok": false,
      "syncing": true
    }
  },
  "note": "abridged"
}
```

11.3 Health and chain identity — `GET /health`

Field	Value
Name	`health` — Health and chain identity
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /health  (also /)`
Purpose	One call that proves WHICH chain this node follows: network, chainId, genesisId, paramsHash, versions, height, head hash, peers count, syncing flag, supply invariant. First call of every integration check.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe; masks nothing sensitive. 500 with `status: degraded` if the supply invariant fails (treat as a stop signal).
Status / error codes	200 OK · 500 `(body status` degraded`)` (supply invariant violated: stop crediting and alert) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`health`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/health"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "status": "ok",
  "coreVersion": "2.0.4",
  "protocolVersion": "2.0.2",
  "minCoreVersion": "2.0.2",
  "network": "devnet",
  "networkId": "obsidian-devnet-1",
  "chainId": 7780,
  "genesisId": "64f5bb13a2432e8d6786ef2dd4d8d86f920f7313",
  "paramsHash": "12211595855ba405b3d70f6161ef9371",
  "height": 4,
  "headHash": "fcf8aaa4012c8c4876403ffcef12d0850b12a836de42dc16f1e76ea6f6242381",
  "peers": 0,
  "syncing": false,
  "supplyOk": true,
  "uptimeSeconds": 20,
  "timestamp": 1792524563
}
```


Example error response [ILLUSTRATIVE] (HTTP 500)


```
{
  "note": "supply invariant violated: stop crediting and alert"
}
```


11.4 Software and protocol version — `GET /version`

Field	Value
Name	`version` — Software and protocol version
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /version`
Purpose	coreVersion, protocolVersion, wireProtocolVersion, minCoreVersion, buildId.
Parameters (type, required)	none

Field	Value
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`versionInfo()` in `obsidian-core/src/version.ts`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/version"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "coreVersion": "2.0.4",
  "protocolVersion": "2.0.2",
  "wireProtocolVersion": 1,
  "minCoreVersion": "2.0.2",
  "buildId": "a77e34a9612474b7"
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.5 Network definition — `GET /network`

Field	Value
Name	`network` — Network definition
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /network`
Purpose	Network name, networkId, chainId, addressHrp (address prefix), p2pMagic, default ports, genesisId, paramsHash, versions, the official `domains` of that network and the trusted-origin policy.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe. It lists domain NAMES, not proof they are live.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`network`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/network"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "network": {},
  "genesisId": "64f5bb13a2432e8d6786ef2dd4d8d86f920f7313",
  "paramsHash": "12211595855ba405b3d70f6161ef9371",
  "coreVersion": "2.0.4",
  "protocolVersion": "2.0.2",
  "domains": [
    "devnet.obsmainnet.us.ci",
    "devnet.mineobs.network"
  ]
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.6 Consensus parameters — `GET /params`

Field	Value
Name	`params` — Consensus parameters
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /params`
Purpose	Consensus constants as the node computes them (gas, block, finality, mining, ONS, ...) plus `paramsHash`. Must be identical on every node of one network.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`params`; `obsidian-core/src/protocol/params.ts`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/params"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "protocolVersion": "2.0.2",
  "paramsHash": "12211595855ba405b3d70f6161ef9371",
  "maximumSupplyObs": "21000000.000000000000000000",
  "gas": {},
  "block": {}
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.7 Genesis document and identity — `GET /genesis`

Field	Value
Name	`genesis` — Genesis document and identity
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /genesis`
Purpose	The genesis document, genesisId (40 hex), genesisHash (64 hex) and the genesis state (including the mining-gate issuer keys that are part of the genesis).
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe. NOTE: genesisId depends on the committed gate keys (see GAP-11): pin the value the operator publishes.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`genesis`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/genesis"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "document": {
    "networkId": "obsidian-devnet-1",
    "chainId": 7780
  },
  "genesisId": "64f5bb13a2432e8d6786ef2dd4d8d86f920f7313",
  "genesisHash": "4204e1b8e6a6b2cf24a5cc82f1f595f21ded64c4eaeff325a595d5a9c16ae223",
  "chainId": 7780,
  "networkId": "obsidian-devnet-1"
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.8 Supply — `GET /supply`

Field	Value
Name	`supply` — Supply
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /supply`
Purpose	Issued supply (seals and OBS), the 21,000,000 OBS cap, supply invariant, mining-pool balance, marketplace escrow, issuance sources. There is NO field named `circulatingSupply` (GAP-12).
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.

Field	Value
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`supply`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/supply"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "totalSupplyObs": "0.000000000000000000",
  "totalSupplySeals": "0",
  "maxSupplyObs": "21000000.000000000000000000",
  "maximumRespected": true,
  "invariantOk": true,
  "genesisIssuedObs": "0.000000000000000000",
  "minedSupplyObs": "0.000000000000000000",
  "poolBalanceObs": "0.000000000000000000",
  "marketEscrowObs": "0.000000000000000000",
  "issuanceSources": [
    "GENESIS_ALLOCATION",
    "MINING_REWARD"
  ]
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.9 Chain status — `GET /status`

Field	Value
Name	`status` — Chain status
Group	Health & identity
Transport	HTTP GET (JSON)
Method and path	`GET /status`
Purpose	Height, headHash, lastBlockTimestamp, supply, mempool size, peers count, syncing, scheduleTier, indexer summary. For an operator-local caller it also carries finalizedHeight / finalizedHash / finalityLag / finalityAvailable / finalityEligibleProducers / finalityQuorum; a caller that is not operator-local (any proxy header, or non-loopback) receives the same document WITHOUT those fields.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe for public exposure since 2.0.3 (finality and peer detail withheld from non-operator callers). Use `lastBlockTimestamp` as protocol time.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`status`, `mayShowFinality`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/status"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "height": 4,
  "headHash": "fcf8aaa4012c8c4876403ffcef12d0850b12a836de42dc16f1e76ea6f6242381",
  "genesisHash": "4204e1b8e6a6b2cf24a5cc82f1f595f21ded64c4eaeff325a595d5a9c16ae223",
  "genesisId": "64f5bb13a2432e8d6786ef2dd4d8d86f920f7313",
  "networkId": "obsidian-devnet-1",
  "chainId": 7780,
  "protocolVersion": "2.0.2",
  "paramsHash": "12211595855ba405b3d70f6161ef9371",
  "mempool": {},
  "peers": 0,
  "syncing": false,
  "supply": "0",
  "lastBlockTimestamp": 1792524559,
  "finalizedHeight": 0,
  "finalityLag": 4,
  "finalityAvailable": false
}
```


Example error response [VERIFIED-DEVNET] (HTTP 429)


```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```


11.10 Finalized checkpoint (operator-local) — `GET /finality`

Field	Value
Name	`finality` — Finalized checkpoint (operator-local)
Group	Finality
Transport	HTTP GET (JSON)
Method and path	`GET /finality`
Purpose	The finalized checkpoint (height, hash, lag, available, eligibleProducers, window, quorum, minProducers) derived from chain state. 404 `ERR_NOT_FOUND` for any caller that is not operator-local.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Not available publicly BY DESIGN. An exchange reads it from ITS OWN node over loopback without proxy headers (or sets `OBSIDIAN_RPC_PUBLIC_PEER_DETAIL=true` on a node that is NOT internet-facing).
Status / error codes	200 OK (operator-local caller) · 404 (other callers) · 404 `ERR_NOT_FOUND` (caller is not operator-local (proxy header present or non-loopback source)) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`finality`, `isLocalCaller`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/finality"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "model": "PRODUCER_DIVERSITY_CHECKPOINT",
  "available": false,
  "reason": "finality is unavailable: 0 established producer(s), at least 2 are required",
  "finalizedHeight": 0,
  "finalizedHash": "4204e1b8e6a6b2cf24a5cc82f1f595f21ded64c4eaeff325a595d5a9c16ae223",
  "headHeight": 4,
  "lag": 4,
  "eligibleProducers": 0,
  "window": 32,
  "quorum": 0,
  "minProducers": 2,
  "producersSeen": 0,
  "candidateHeight": 0,
  "derivedFrom": "chain state only: a block is final once enough distinct established producers have built on it",
  "assumptions": "checkpoint (economic) finality, not BFT: it holds while fewer than one third of the established
      ↪ producers are faulty, and a new node must ob…(truncated)"
}
```


Example error response [VERIFIED-DEVNET] (HTTP 404)


```
{
  "error": "not found",
  "code": "ERR_NOT_FOUND",
  "path": "/finality"
}
```


11.11 List blocks — `GET /blocks?limit={1..100}&from={height}`

Field	Value
Name	`blocks` — List blocks
Group	Blocks & transactions
Transport	HTTP GET (JSON)
Method and path	`GET /blocks?limit={1..100}&from={height}`
Purpose	Recent blocks (default 10) or, with `from`, canonical entries from that height upward: hash, height, prevHash, timestamp, txCount, producer (masked), size. This is the scanner's cursor: walk heights, compare prevHash, detect reorgs.
Parameters (type, required)	`limit` (query, integer 1–100, optional) — page size; default 10; `from` (query, integer ≥ 0, optional) — first height to return (switches to canonical range mode)
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe; producer addresses are masked.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`blocks`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/blocks?limit=<1..100>&from=<height>"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "blocks": [
    {
      "hash": "8defdc61e88d4561756f8a41d0da833e65ad0832ce2cc06dcfc38e3d3fbd6994",
      "height": 5,
      "prevHash": "fcf8aaa4012c8c4876403ffcef12d0850b12a836de42dc16f1e76ea6f6242381",
      "timestamp": 1792524564,
      "txCount": 1,
      "producer": "dobs1k00q9…asfgvr",
      "size": 1192
    },
    {
      "hash": "663c2d068f37c109b9aaa603854b5d84e0c7910d899c9400b04bc7022b2d85b2",
      "height": 6,
      "prevHash": "8defdc61e88d4561756f8a41d0da833e65ad0832ce2cc06dcfc38e3d3fbd6994",
      "timestamp": 1792524569,
      "txCount": 1,
      "producer": "dobs1k00q9…asfgvr",
      "size": 981
    }
  ]
}
```


Example error response [VERIFIED-DEVNET] (HTTP 429)


```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```


11.12 Block by height or hash — `GET /block/{height|hash}`

Field	Value
Name	`block` — Block by height or hash
Group	Blocks & transactions
Transport	HTTP GET (JSON)
Method and path	`GET /block/{height|hash}`
Purpose	Block header, summary, `confirmations`, and the transaction list (id, type, FULL sender, nonce, gas, validUntil, size) plus `events` for that height. The transaction list has NO recipient and NO amount (GAP-02); the PAYMENT events carry from/to/amount/gas/memo but only the most recent 200 events are searched (GAP-03).
Parameters (type, required)	`height|hash` (path, integer or 64-hex, REQUIRED) — block height (digits) or block hash
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe. Full SENDER addresses are public here; recipients are not.
Status / error codes	200 OK · 404 `ERR_NOT_FOUND` (no canonical block at that height / hash) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`block`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/block/<height>"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "summary": "…",
  "header": "…",
  "transactions": [
    {
      "id": "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9",
      "type": "PAYMENT",
      "sender": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2",
      "nonce": 1,
      "gas": "2500000000000000",
      "validUntil": 1792525165,
      "size": 375
    }
  ],
  "events": [
    {
      "type": "PAYMENT",
      "height": 6,
      "txId": "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9",
      "data": {
        "from": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2",
        "to": "dobs1nu6lkws62au6g80u9vgv0akp2qj53gef30fxme",
        "amount": "12500000000000000000",
        "gas": "2500000000000000",
        "memo": "dep-1001"
      },
      "blockHash": "663c2d068f37c109b9aaa603854b5d84e0c7910d899c9400b04bc7022b2d85b2"
    }
  ],
  "confirmations": 1,
  "hash": "663c2d068f37c109b9aaa603854b5d84e0c7910d899c9400b04bc7022b2d85b2"
}
```


Example error response [ILLUSTRATIVE] (HTTP 404)


```
{
  "error": "no canonical block at that height / hash",
  "code": "ERR_NOT_FOUND"
}
```


11.13 Transaction lookup — `GET /tx/{txid}`

Field	Value
Name	`tx` — Transaction lookup
Group	Blocks & transactions
Transport	HTTP GET (JSON)
Method and path	`GET /tx/{txid}`
Purpose	Status of one transaction by its 64-hex id. Mempool → `{status:"PENDING"}`. Included → the index record plus `confirmations` and `confirmed:true`. Sender and recipient are masked (first 10 … last 6 characters); `amount` (seals, string), `gas`, `height`, `blockHash`, `index`, `timestamp` are exact. Not found → 404 (never seen, expired, or rolled back by a reorg).
Parameters (type, required)	`txid` (path, 64 hex, REQUIRED) — transaction id (computed by the signer before broadcast)
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe. Masked counterparties cannot be used to identify a deposit unambiguously: use JSON-RPC `gettransaction` on your own node (GAP-01).
Status / error codes	200 OK · 400 `ERR_MALFORMED` (txid is not 64 hex) · 404 `ERR_NOT_FOUND` (unknown to this node (not a proof it never existed)) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`transaction`, `serializeTransaction`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/tx/<txid>"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "txId": "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9",
  "height": 6,
  "blockHash": "663c2d068f37c109b9aaa603854b5d84e0c7910d899c9400b04bc7022b2d85b2",
  "index": 0,
  "type": 1,
  "sender": "dobs1mfca0…r4hhl2",
  "gas": "2500000000000000",
  "timestamp": 1792524569,
  "status": "INCLUDED",
  "recipient": "dobs1nu6lk…30fxme",
  "amount": "12500000000000000000",
  "confirmations": 1,
  "confirmed": true
}
```


Example error response [VERIFIED-DEVNET] (HTTP 404)


```
{
  "error": "transaction not found",
  "code": "ERR_NOT_FOUND"
}
```


11.14 Address history (indexer) — `GET /address/{address}?limit={1..100}`

Field	Value
Name	`address` — Address history (indexer)
Group	Blocks & transactions
Transport	HTTP GET (JSON)
Method and path	`GET /address/{address}?limit={1..100}`
Purpose	Transactions that involve the address (as sender or recipient), newest first, up to `limit` (max 100, no cursor: the 100 newest only, see GAP-04), plus mining claims and protocol credits. Counterparties masked; no balance. Indexer data: rebuildable and never authoritative.
Parameters (type, required)	`address` (path, bech32 address of THIS network, REQUIRED) — lowercase (ALL-UPPERCASE is canonicalised; mixed case → 400); `limit` (query, integer ≤ 100, optional) — default 25
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe; masked.
Status / error codes	200 OK · 400 `ERR_BAD_ADDRESS` (invalid, or prefix of another network) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`addressHistory`; `obsidian-core/src/indexer/indexer.ts`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/address/<address>?limit=<1..100>"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "address": "dobs1nu6lk…30fxme",
  "maskNote": "Public explorer output: counterparties and this address are partially masked. Balances are never
      ↪ exposed.",
  "transactions": [
    {
      "txId": "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9",
      "height": 6,
      "blockHash": "663c2d068f37c109b9aaa603854b5d84e0c7910d899c9400b04bc7022b2d85b2",
      "index": 0,
      "type": 1,
      "sender": "dobs1mfca0…r4hhl2",
      "gas": "2500000000000000",
      "timestamp": 1792524569,
      "status": "INCLUDED",
      "recipient": "dobs1nu6lk…30fxme",
      "amount": "12500000000000000000"
    }
  ],
  "miningClaims": [],
  "credits": [],
  "creditsNote": "Credits are OBS the protocol itself paid into this address inside a block (treasury share of ONS
      ↪ fees, Node Runner rewards). They are not tr…(truncated)",
  "counts": {
    "transactions": 1,
    "miningClaims": 0,
    "credits": 0
  }
}
```


Example error response [ILLUSTRATIVE] (HTTP 400)


```
{
  "error": "invalid, or prefix of another network",
  "code": "ERR_BAD_ADDRESS"
}
```


11.15 Mempool snapshot — `GET /mempool`

Field	Value
Name	`mempool` — Mempool snapshot
Group	Blocks & transactions
Transport	HTTP GET (JSON)
Method and path	`GET /mempool`
Purpose	Pending transaction count, bytes and the first 100 entries (txId, type, masked sender, gas).
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe; masked senders.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`mempool`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/mempool"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "size": 0,
  "bytes": 0,
  "transactions": []
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.16 Next nonce — `GET /wallet/{address}/next-nonce`

Field	Value
Name	`nextnonce` — Next nonce
Group	Wallet / account
Transport	HTTP GET (JSON)
Method and path	`GET /wallet/{address}/next-nonce`
Purpose	The nonce the next transaction from this address must carry (account nonce at the head state), and the head height it was read at. Does not count mempool transactions.
Parameters (type, required)	`address` (path, bech32, REQUIRED) — sender address
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe.
Status / error codes	200 OK · 400 `ERR_BAD_ADDRESS` (invalid address) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`walletNonce`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/wallet/<address>/next-nonce"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "address": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2",
  "nextNonce": 3,
  "atHeight": 8
}
```

Example error response [ILLUSTRATIVE] (HTTP 400)

```
{
  "error": "invalid address",
  "code": "ERR_BAD_ADDRESS"
}
```

11.17 Balance — `POST /wallet/balance`

Field	Value
Name	`balance` — Balance
Group	Wallet / account
Transport	HTTP POST (JSON)
Method and path	`POST /wallet/balance`
Purpose	Authoritative account state at the node's HEAD block (0 confirmations): `balanceSeals`/`spendableSeals` (spendable now), `lockedSeals` (marketplace escrow), `totalSeals`, `nonce`, `txCount`, `receivedObs`, `sentObs`, `createdAtHeight`, `atHeight`. An address that never appeared on chain returns zeros (not an error). There is no 'pending' or 'confirmed-only' balance: an exchange computes its own credited balance from confirmed deposits.
Parameters (type, required)	`address` (body, string, REQUIRED) — bech32 address of this network (ALL-UPPERCASE accepted and lowercased)
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).

Field	Value
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Reachable through the project's interface proxy too; anyone who knows an address can read its balance (by design).
Status / error codes	200 OK · 400 `ERR_BAD_ADDRESS` (invalid address or another network's prefix) · 400 `ERR_MALFORMED` (body is not a JSON object) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`walletBalance`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/wallet/balance" -H 'content-type: application/json' \
  -d '{"address": "dobs1nu6lkws62au6g80u9vgv0akp2qj53gef30fxme"}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)


```
{
  "address": "dobs1nu6lkws62au6g80u9vgv0akp2qj53gef30fxme",
  "balanceObs": "12.500000000000000000",
  "balanceSeals": "12500000000000000000",
  "spendableObs": "12.500000000000000000",
  "spendableSeals": "12500000000000000000",
  "lockedObs": "0.000000000000000000",
  "lockedSeals": "0",
  "totalObs": "12.500000000000000000",
  "totalSeals": "12500000000000000000",
  "nonce": 0,
  "txCount": 0,
  "receivedObs": "12.500000000000000000",
  "sentObs": "0.000000000000000000",
  "createdAtHeight": 6,
  "mining": null,
  "names": [],
  "atHeight": 6,
  "custodial": false
}
```


Example error response [VERIFIED-DEVNET] (HTTP 400)


```
{
  "error": "invalid wallet address",
  "code": "ERR_BAD_ADDRESS"
}
```


11.18 Send quote (gas + sufficiency + nonce + protocol time) — `POST /wallet/quote`

Field	Value
Name	`quote` — Send quote (gas + sufficiency + nonce + protocol time)
Group	Wallet / account
Transport	HTTP POST (JSON)
Method and path	`POST /wallet/quote`
Purpose	For an address and an amount: balance, gas, total (amount + gas), `sufficient`, `nextNonce`, and `protocolTime` (use it to compute validUntil). The most useful pre-flight call for a withdrawal.
Parameters (type, required)	`address` (body, string, REQUIRED) — sender; `amountObs` (body, decimal string, optional) — OBS, up to 18 decimals; default 0; `usd` (body, decimal string, optional) — optional USD quote; needs a usable oracle (NOT relevant to exchanges)
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe.

Field	Value
Status / error codes	200 OK · 400 `ERR_BAD_ADDRESS` (invalid address) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`walletQuote`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/wallet/quote" -H 'content-type: application/json' \
  -d '{"address": "dobs1npnfngv5ekpjh2ayne90fues5qfdm40w5yklpu", "amountObs": "1"}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "address": "dobs1npnfngv5ekpjh2ayne90fues5qfdm40w5yklpu",
  "balanceObs": "0.000000000000000000",
  "amountObs": "1.000000000000000000",
  "gasObs": "0.000200000000000000",
  "totalObs": "1.000200000000000000",
  "sufficient": false,
  "usdQuoteObs": null,
  "obsPriceUsd": "$0",
  "oracleUsable": false,
  "nextNonce": 0,
  "protocolTime": 1792524575
}
```

Example error response [ILLUSTRATIVE] (HTTP 400)

```
{
  "error": "invalid address",
  "code": "ERR_BAD_ADDRESS"
}
```

11.19 Gas (fee) quote — `POST /tx/gas`

Field	Value
Name	`gas` — Gas (fee) quote
Group	Fees
Transport	HTTP POST (JSON)
Method and path	`POST /tx/gas`
Purpose	The exact gas a PAYMENT of `amountObs` must carry. Gas is a protocol formula, not a market: gas = clamp(floor(amount × 2 / 10000), 0.000001 OBS, 0.01 OBS), paid by the sender IN ADDITION to the amount, to the Mining Pool.
Parameters (type, required)	`amountObs` (body, decimal string, REQUIRED) — OBS amount; `usd` (body, decimal string, optional) — optional
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe.
Status / error codes	200 OK · 400 `ERR_MALFORMED` (amount not a decimal with ≤ 18 places) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`gasQuote`; `transactions/helpers.ts expectedGas`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/tx/gas" -H 'content-type: application/json' \
  -d '{"amountObs": "12.5"}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "amountObs": "12.500000000000000000",
  "gasObs": "0.002500000000000000",
  "usdObs": null,
  "usdGasObs": null,
  "obsPriceUsd": "$0",
  "gasFormula": "gas = min(floor(amount * 2 / 10000), 0.01 OBS), destination = MINING_POOL"
}
```

Example error response [ILLUSTRATIVE] (HTTP 400)

```
{
  "error": "amount not a decimal with ≤ 18 places",
  "code": "ERR_MALFORMED"
}
```

11.20 Encode / decode a transaction BODY — `POST /tx/encode`

Field	Value
Name	`encode` — Encode / decode a transaction BODY
Group	Transactions
Transport	HTTP POST (JSON)
Method and path	`POST /tx/encode`
Purpose	Server-side helper that turns `{type, payload}` into canonical body hex, or (with `body`) decodes a body hex. It does NOT sign and does NOT build the envelope. For PAYMENT the payload is `{to, amount (seals, string), memo?}`. Prefer local encoding; use this as a cross-check.
Parameters (type, required)	`type` (body, string, REQUIRED) — e.g. `PAYMENT`; `payload` (body, object, optional) — encode mode; `body` (body, hex, optional) — decode mode
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Safe; stateless.
Status / error codes	200 OK · 400 `ERR_MALFORMED` (payload missing or invalid (e.g. no `amount` in seals)) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`encodeTx`, `encodeBodyFor`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/tx/encode" -H 'content-type: application/json' \
  -d '{"type": "PAYMENT", "payload": {"to": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2", "amount":
      ↪ "12500000000000000000", "memo": "dep-1001"}}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "body": "0000002b646f6273316d6663613063726c7a7174393674367236396e7334346365677936393346d7174723468686c320000000000000
      ↪ 000ad78ebc5ac62000000000000086465702d31303031",
  "type": "PAYMENT"
}
```

Example error response [ILLUSTRATIVE] (HTTP 400)

```
{
  "error": "payload missing or invalid (e.g. no `amount` in seals)",
  "code": "ERR_MALFORMED"
}
```

11.21 Simulate a signed transaction — `POST /tx/simulate`

Field	Value
Name	`simulate` — Simulate a signed transaction
Group	Transactions
Transport	HTTP POST (JSON)
Method and path	`POST /tx/simulate`
Purpose	Dry-run a signed transaction hex against the head state: `valid`, `error`, `projectedStateRoot`, `events`. No admission, no broadcast. NOTE: it reports state-dependent validity only; use /tx/submit for the full admission screen.
Parameters (type, required)	`tx` (body, hex, REQUIRED) — signed transaction bytes, hex
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Does work per call (copies state); keep the rate limit on.
Status / error codes	200 OK · 400 `ERR_MALFORMED` (not hex / cannot decode) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`simulate`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/tx/simulate" -H 'content-type: application/json' \
  -d '{"tx": "<signed-tx-hex>"}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "valid": true,
  "error": null,
  "projectedStateRoot": "596f0fd77cda40fe4486fc4d471349fda35c106ace0bba0e029df8ee08317b98",
  "events": [
    {
      "type": "PAYMENT",
      "data": {
        "from": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2",
        "to": "dobs1nu6lkws62au6g80u9vgv0akp2qj53gef30fxme",
        "amount": "12500000000000000000",
        "gas": "2500000000000000",
        "memo": "dep-1001"
      }
    }
  ]
}
```

Example error response [ILLUSTRATIVE] (HTTP 400)

```
{
  "error": "not hex / cannot decode",
  "code": "ERR_MALFORMED"
}
```

11.22 Broadcast a signed transaction — `POST /tx/submit`

Field	Value
Name	`submit` — Broadcast a signed transaction
Group	Transactions
Transport	HTTP POST (JSON)
Method and path	`POST /tx/submit`

Field	Value
Purpose	Admit a signed transaction to the node's mempool and gossip it. `accepted:true` means ADMITTED, not included and not final. Identical resubmission while pending → 200 `duplicate:true` (idempotent). Another tx with the same sender+nonce → 409 `ERR_NONCE_PENDING`. A wrong nonce (including nonce+1 while one is pending) → 400 `ERR_REJECTED ERR_BAD_NONCE`.
Parameters (type, required)	`tx` (body, hex, REQUIRED) — signed transaction bytes (≤ 16 KiB), hex
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	A node with `rpcAllowSubmit=false` refuses writes. The project's interface proxy does NOT expose /tx/submit to third parties (only a gated mining claim).
Status / error codes	200 OK · 400 `ERR_MALFORMED` (not hex / cannot decode) · 400 `ERR_WRONG_CHAIN_ID` (tx chainId ≠ node chainId) · 400 `ERR_REJECTED` (screening or simulation failed; text starts with the cause: ERR_BAD_NONCE, ERR_BAD_GAS, ERR_AMOUNT_ZERO (below minimum), ERR_EXPIRED, ERR_BAD_ADDRESS, ERR_INSUFFICIENT_FUNDS, ERR_BAD_SIGNATURE, ERR_MALFORMED) · 403 `ERR_FORBIDDEN` (`rpcAllowSubmit` is false on this node) · 409 `ERR_NONCE_PENDING` (different tx with same sender+nonce is pending) · 429 `ERR_RATE_LIMITED` (rate limit or mempool admission refused (full / sender quota 64)) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`submitTransaction`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/tx/submit" -H 'content-type: application/json' \
  -d '{"tx": "<signed-tx-hex>"}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "accepted": true,
  "txId": "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9",
  "type": "PAYMENT",
  "sender": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2",
  "stateRootPreview": "596f0fd77cda40fe4486fc4d471349fda35c106ace0bba0e029df8ee08317b98",
  "note": "The transaction is admitted and broadcast for a scheduled producer to consider. Admission does not
      ↪ guarantee inclusion, and inclusion is not…(truncated)"
}
```

Example error response [VERIFIED-DEVNET] (HTTP 409)

```
{
  "error": "this account already has a transaction with nonce 0 waiting for the next block (253d0343a968e9f1…); wait
      ↪ for it to be confirmed, then send again",
  "code": "ERR_NONCE_PENDING",
  "pendingTxId": "253d0343a968e9f15071473debccacfa61f9007e78a5f510f5b9b15e3ec00d1f",
  "txId": "fa61e43e9e90b37a5e6e8fe4d43f3a82ed6c9f58eb24bb73b3ddeaea27134469"
}
```

11.23 JSON-RPC 2.0 endpoint — `POST /rpc`

Field	Value
Name	`jsonrpc` — JSON-RPC 2.0 endpoint
Group	JSON-RPC
Transport	HTTP POST (JSON-RPC 2.0)
Method and path	`POST /rpc`

Field	Value
Purpose	Compatibility façade. Methods (all verified in source): `getstatus`, `getfinality` (operator-local, else null), `getblock {height}` (header only), `getblocks {from, limit≤500}` (hashes), `gettransaction {txId}` (UNMASKED sender, recipient, amount, gas, height, blockHash, index, timestamp; no confirmations, no memo; no PENDING state), `getbalance {address}` (balanceObs, nonce), `getminingstatus`, `getparams`, `getoracle`, `getnames`, `getpeers`/`getnodes` (operator-local, else empty), `submittransaction` (always an error: use POST /tx/submit). ALL JSON-RPC errors are HTTP 200 with `error:{code:-32000, message, data:{code:"ERR_METHOD"}}`.
Parameters (type, required)	`jsonrpc` (body, "2.0", optional) — accepted, not enforced; `method` (body, string, REQUIRED) — see list; `params` (body, object (not array), optional) — named parameters only; `id` (body, number|string, optional) — echoed
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	`gettransaction` is the only route that returns unmasked recipients: that is why deposit detection should run against YOUR OWN node. Do not expose /rpc of that node publicly.
Status / error codes	200 OK · 200 `-32000 / ERR_METHOD` (any method error, including unknown method and bad params) · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`jsonRpc`, `invokeJsonRpc`

Example request [VERIFIED-DEVNET]

```
curl -s -X POST "$NODE/rpc" -H 'content-type: application/json' \
  -d '{"jsonrpc": "2.0", "id": 1, "method": "gettransaction", "params": {"txId":
      ↪ "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9"}}'
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "txId": "639347305cf18c82184b01208f310ea8795b67e5316e9174803b49cb08ef56e9",
    "height": 6,
    "blockHash": "663c2d068f37c109b9aaa603854b5d84e0c7910d899c9400b04bc7022b2d85b2",
    "index": 0,
    "type": 1,
    "sender": "dobs1mfca0crlzqt96t6r69ns44cegy694mqtr4hhl2",
    "gas": "2500000000000000",
    "timestamp": 1792524569,
    "status": "INCLUDED",
    "recipient": "dobs1nu6lkws62au6g80u9vgv0akp2qj53gef30fxme",
    "amount": "12500000000000000000"
  }
}
```

Example error response [VERIFIED-DEVNET] (HTTP 200)

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "error": {
    "code": -32000,
    "message": "invalid address",
    "data": {
      "code": "ERR_METHOD"
    }
  }
}
```

11.24 Peers (operator-local) — `GET /peers`

Field	Value
Name	`peers` — Peers (operator-local)
Group	Network
Transport	HTTP GET (JSON)
Method and path	`GET /peers`
Purpose	Peer table for the operator. A caller that is not operator-local gets counts only (`detail: counts-only`, empty `connected`/`known`). `/nodes` behaves the same.
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	Peer addresses are withheld from non-operator callers since 2.0.2/2.0.3.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`peers`, `publicP2pStatus`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/peers"
```

Example success response [VERIFIED-DEVNET transcript] (HTTP 200)

```
{
  "p2p": {
    "listening": true,
    "peers": {
      "connected": 0,
      "inbound": 0,
      "outbound": 0,
      "pendingHandshakes": 0,
      "known": 0,
      "height": 0
    },
    "wireProtocol": 1,
    "lastError": null
  },
  "connected": [],
  "known": [],
  "detail": "counts-only",
  "note": "Peer addresses are shown only to a caller on this machine. The operator can publish them with
      ↪ OBSIDIAN_RPC_PUBLIC_PEER_DETAIL=true."
}
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.25 Prometheus metrics — `GET /metrics`

Field	Value
Name	`metrics` — Prometheus metrics
Group	Network
Transport	HTTP GET (text)
Method and path	`GET /metrics`
Purpose	Prometheus exposition: height, finalized height, peers, mempool, etc. No address or balance. The node applies NO caller check to it (verified: served to a proxied caller), so the reverse proxy MUST block it from the internet.

Field	Value
Parameters (type, required)	none
Authentication	None. The node RPC has no API key or login. Protect it by network placement (see §12).
Rate limits / timeouts	Per client IP: 240 requests/minute by default (`rpcRateLimitPerMinute`, 0 disables); over the limit → HTTP 429 `ERR_RATE_LIMITED` (no `Retry-After` header is sent: see GAP-14). Timeouts: headers 10 s, request 15 s, socket 20 s.
Network restrictions	All four networks. Only the base URL, port and address prefix differ (mainnet 8630 / `obs`, testnet 18630 / `tobs`, staging 28630 / `sobs`, devnet 38630 / `dobs`).
Public-exposure safety	PRIVATE. Includes `obsidian_finalized_height`, so exposing it publicly leaks the finality value the project withholds from /status.
Status / error codes	200 OK · plus the common statuses above
Evidence	VERIFIED (source) + VERIFIED (devnet run)
Source	`metrics`

Example request [VERIFIED-DEVNET]

```
curl -s "$NODE/metrics"
```

Example success response [ILLUSTRATIVE (shortened)] (HTTP 200)

```
# HELP obsidian_chain_height Height of the node's current head block.
# TYPE obsidian_chain_height gauge
obsidian_chain_height{network="devnet",chain_id="7780"} 47
…
```

Example error response [VERIFIED-DEVNET] (HTTP 429)

```
{
  "error": "rate limit exceeded",
  "code": "ERR_RATE_LIMITED"
}
```

11.26 Capabilities that do NOT exist

Capability	Status	What to do instead
Wallet / address creation endpoint	**NOT IMPLEMENTED**	By design the node never sees keys. Create wallets offline: CLI `obsidian-core wallet new --network mainnet` (one wallet, index 0) or the core `deriveWallet()` function (HD, many addresses). See §6.
Address validation endpoint	**NOT IMPLEMENTED**	Validate locally (bech32 checksum, prefix, 20-byte payload; reference code in examples/). `POST /wallet/balance` and `GET /address/…` return 400 `ERR_BAD_ADDRESS` as a server-side cross-check.
Server-side transaction build/sign endpoint	**NOT IMPLEMENTED**	Non-custodial by design. Build and sign locally; `/tx/encode` only encodes a body.
Transaction receipt endpoint	**NOT IMPLEMENTED**	Use `GET /tx/{txid}` (status + height + blockHash + confirmations) and `/block/{h}` events.
Unmasked REST transaction / address history	**NOT IMPLEMENTED**	REST masks counterparties. Unmasked data: JSON-RPC `gettransaction` and the sender in `/block/{h}`.
Pending / unconfirmed balance	**NOT IMPLEMENTED**	`/wallet/balance` is the head state only. Track unconfirmed deposits yourself from `/mempool` or your own scanner.
Paginated history (cursor / offset)	**NOT IMPLEMENTED**	`/address/…` returns at most the 100 newest records. Scan blocks instead (§9).
WebSocket / webhook / push notifications	**NOT IMPLEMENTED**	Poll `/blocks?from=` (cheap) — no subscription mechanism exists in the node.
Public finality value	**NOT AVAILABLE TO PUBLIC CALLERS**	Operator-local only since 2.0.3. Run your own node, or use a confirmation-count policy.
Batch / multi-recipient transfer	**NOT IMPLEMENTED**	A PAYMENT has exactly one recipient.

Capability	Status	What to do instead
Tag/memo-addressed deposit lookup	**NOT IMPLEMENTED**	A memo exists (≤ 256 bytes) but cannot be searched, and the body memo is only retrievable from recent block events. Use one address per customer.

12. Running your own node and exposing it safely

12.1 Why the exchange should run its own node

1. Finality values are visible only to an operator-local caller (§5.6).
2. Unmasked recipients exist only on JSON-RPC `gettransaction`, which should not be public.
3. No authentication and per-IP limits only: a shared public node is a shared rate limit and a single point of failure.
4. You pin the genesis identity yourself and validate every block.
5. No push mechanism: you need a low-latency, high-rate local read path.

12.2 Deployment outline [VERIFIED-SRC] (commands exist; mainnet run not performed here)

```
# 1. build from the release tag you pinned (v2.0.4) — Node.js ≥ 20.10
cd obsidian-core && npm ci && npx tsc -p .
# 2. adopt the network, pinning the genesis id the operator published (placeholder)
node dist/index.js join --network mainnet --peer https://<operator-confirmed-node> --genesis-id <40-hex-genesisId>
# 3. run a FOLLOWER (validates, does not produce blocks); RPC stays on loopback
OBSIDIAN_KEYSTORE_PASSPHRASE=<from your secret store> \
OBSIDIAN_SEED_NODES=<operator-confirmed host:port,host:port> \
node dist/index.js start --network mainnet --no-mine --rpc-host 127.0.0.1 --data-dir /var/lib/obsidian/mainnet/node
node dist/index.js doctor --network mainnet     # diagnoses config, ports, keys and a running node
```

Defaults that matter: `rpcHost 127.0.0.1`, `rpcPort 8630`, `rpcAllowSubmit true`, `rpcRateLimitPerMinute 240`, `p2pHost 0.0.0.0`, `p2pPort 8631`, `maxPeers 16`. Open **only P2P 8631/tcp inbound** (optional but recommended for connectivity); keep RPC 8630 on loopback or a private subnet. **The seed list and the `join` peer must come from the operator [OPERATOR] (GAP-08).** Do not point `--genesis-id` at a value you did not receive out of band.

12.3 Exposure rules (RPC is unauthenticated)

Route group	Public?	Why
`/livez`, `/health`, `/version`, `/network`, `/params`, `/genesis`, `/supply`, `/blocks`, `/block/*`, `/tx/{id}`, `/address/*`, `/mempool`	acceptable behind TLS + rate limit	public chain data, counterparties masked
`/status`, `/readyz`	acceptable	finality and peer detail withheld from non-operator callers
`POST /wallet/balance`, `/wallet/quote`, `/wallet/*/next-nonce`	acceptable but reveals balances of known addresses	by design
`POST /tx/submit`, `/tx/simulate`	only to your own services	simulation copies state; submit spends your rate budget
`POST /rpc`	**private**	returns unmasked transactions and, to a local caller, peers/finality
`/metrics`, `/finality`, `/peers`, `/nodes`, `/producers`	**private**	`/metrics` has **no caller check at the node** and exposes the finalized height (GAP-13)

A reverse proxy in front of the node that adds `X-Forwarded-*` or `Via` headers makes the node treat the caller as non-operator (finality/peer detail withheld). For internal exchange services either call the node on loopback **without** such headers, or set `OBSIDIAN_RPC_PUBLIC_PEER_DETAIL=true` on a node whose RPC port is **not** reachable from the internet.

13. Security

Each item is labelled **[PROJECT]** (a mechanism that exists in the code) or **[EXCHANGE]** (a recommendation for the exchange).

13.1 Key custody

- **[PROJECT]** Non-custodial: the node and the project's platform never receive wallet keys; the platform stores only public addresses (`docs/wallet-custody.md`).
- **[EXCHANGE]** Phrase generated offline; split custody (e.g. 2-of-3 shares) for cold storage; hot-wallet key in an HSM/KMS or an isolated signer service with a narrow API (`sign(withdrawalId)`), never in the application server.
- **[EXCHANGE]** Hot wallet holds a small float; cold wallet receives sweeps. Deposit addresses hold no keys online (xpub).

13.2 Local signing and keystore

- **[PROJECT]** All signing is local (browser wallet code is the same module as the node). Node identity keystore: AES-256-GCM, scrypt N=2^17,r=8,p=1 (`crypto/keystore.ts`). Browser vault: PBKDF2-SHA256 600,000 iterations + AES-256-GCM.
- **[EXCHANGE]** The signer verifies what it signs: recipient from your DB, amount, gas = formula, chainId = 7777, protocol version = your pin; refuse anything else.

13.3 Transport and perimeter

- **[PROJECT]** Plain HTTP at the node; TLS terminates at your proxy; body ≤ 512 KiB; header/request/socket timeouts 10/15/20 s; origin check; per-IP rate limit.
- **[EXCHANGE]** TLS 1.2+ on any externally reachable proxy; firewall RPC to loopback/VPC; allow P2P 8631 only; deny `/metrics`, `/rpc`, `/peers`, `/finality` at the proxy; per-client rate limits and IP allow-lists at the proxy (the node has none beyond per-IP).

13.4 Replay and duplicate protection

- **[PROJECT]** chainId and nonce signed; txid deterministic; duplicate submit is a no-op or a refusal; ERR_NONCE_PENDING prevents two competing transactions in the mempool.
- **[EXCHANGE]** unique constraints on `txid` and `withdrawalId`; refuse a second signing request for the same withdrawal.

13.5 Withdrawal approvals and limits

[EXCHANGE] dual control above a threshold; velocity and daily limits per customer and per hot address; address allow-list/cool-down after changes; manual review for the first withdrawal to a new address; kill-switch that stops signing without stopping deposits.

13.6 Backup and disaster recovery

[EXCHANGE] offline encrypted backup of phrase shares (tested restore, in a drill); signer config; database with deposits/withdrawals/ledger; node data directory is re-syncable but keep a snapshot to shorten recovery. **Never** restore a node data directory from a different network (`networkId` mismatch is refused by the node); never roll back a data directory (the node refuses to start if finality regressed).

13.7 Monitoring and alerts

`/readyz` ≠ 200; `/health.supplyOk` false; height not advancing for > 60 s (12 target intervals); `syncing:true`; peers = 0; genesisId or chainId change; reorg ≥ 1 on a credited deposit; withdrawal in `BROADCAST` past `validUntil`; hot-wallet balance below float; reconciliation mismatch; 429s from your own node. **[PROJECT]** `/metrics` (Prometheus) exists for this.

13.8 Incident response

[EXCHANGE] Freeze withdrawals (kill-switch); keep deposits scanning; capture node logs and the chain state; compare your node with a second independently operated node via `GET /health` (height/headHash) and `/block/{h}.hash`; do not resume until both agree and the cause is known; rotate hot-wallet keys by creating a new wallet and sweeping (no key rotation exists in-protocol).

13.9 Upgrades

[PROJECT] Protocol identity is part of the handshake: peers with a different protocol version, paramsHash or genesisId are refused; `docs/mainnet-launch.md` states a protocol activation is **genesis-bound** with "no rolling upgrade or automatic state migration" (GAP-19). **[EXCHANGE]** subscribe to operator announcements; stage every new version on testnet; pin `coreVersion`/`protocolVersion` in config and refuse to sign if `/health` differs; schedule maintenance windows; never auto-update the signer.

14. Examples

Every example carries its evidence tag. Hosts are **placeholders** (`NODE`); secrets are placeholders (`<…>`); devnet examples use the devnet address prefix `dobs`. Files are in `docs/exchange/examples/`.

14.1 cURL [VERIFIED-DEVNET] (replace `NODE` with your own node; mainnet RPC default port is 8630)

```
NODE=http://127.0.0.1:8630          # your own node, loopback

curl -s $NODE/health                # identity: network, chainId, genesisId, paramsHash, height
curl -s $NODE/readyz                # 200 ready | 503 not-ready
curl -s -X POST $NODE/wallet/balance -H 'content-type: application/json' -d '{"address":"obs1…"}'
curl -s -X POST $NODE/wallet/quote   -H 'content-type: application/json' -d '{"address":"obs1…","amountObs":"12.5"}'
curl -s -X POST $NODE/tx/gas         -H 'content-type: application/json' -d '{"amountObs":"12.5"}'
curl -s -X POST $NODE/tx/submit      -H 'content-type: application/json' -d '{"tx":"<signed-tx-hex>"}'
curl -s $NODE/tx/<txid>
curl -s "$NODE/blocks?from=<height>&limit=100"
curl -s $NODE/block/<height>
curl -s -X POST $NODE/rpc -H 'content-type: application/json' \
     -d '{"jsonrpc":"2.0","id":1,"method":"gettransaction","params":{"txId":"<txid lowercase>"}}'
```

14.2 Raw JSON: a transaction lookup and the same transaction through JSON-RPC [VERIFIED-DEVNET]

```
// GET /tx/{txid}  — masked counterparties
{"txId":"590d…322d","height":7,"blockHash":"8792…b928","index":0,"type":1,
 "sender":"dobs1ekf0z…ht34ut","recipient":"dobs1xqvd9…4z2726","amount":"12500000000000000000",
 "gas":"2500000000000000","timestamp":1792524155,"status":"INCLUDED","confirmations":1,"confirmed":true}

// POST /rpc gettransaction — unmasked
{"jsonrpc":"2.0","id":1,"result":{"txId":"590d…322d","height":7,"blockHash":"8792…b928","index":0,"type":1,
 "sender":"dobs1ekf0zmcx6rl7zvz2pnkq7tft5r99kcr9ht34ut","recipient":"dobs1xqvd9ewasu9f8qdlwr36puczran2c2fe4z2726",
 "amount":"12500000000000000000","gas":"2500000000000000","timestamp":1792524155,"status":"INCLUDED"}}
```

14.3 Node.js [VERIFIED-DEVNET]

- `examples/node/withdraw.mjs` — derive, quote, build, sign, simulate, broadcast, poll inclusion. Ran on devnet: tx accepted and included at height 35 (txid in the run output), and it **refuses to run if the node's chain id is not the one you expect** (tested: pointing it at devnet while expecting 7777 aborts).
- `examples/node/xpub-deposit-addresses.mjs` — offline `export`, online `derive`, and `selftest` (**PASS**).
- `verification/devnet-verify.mjs` — the full harness (derive, scan, deposit, sweep, error matrix).

Excerpt (the signing core, from `withdraw.mjs`):

```
const gas = helpers.expectedGas(amount);                    // integer math, exact
const tx = enc.signTransaction({
  sender: kp.address, privateKeyHex: kp.privateKey, publicKeyHex: kp.publicKey,
  chainId: net.network.chainId, protocolVersion: PROTOCOL_VERSION,
  nonce: q.body.nextNonce, type: types.TxType.PAYMENT, gas,
  body: pay.encodePaymentBody({ to, amount, memo }), validUntil: q.body.protocolTime + 600,
});
const hex = Buffer.from(enc.encodeSignedTx(tx)).toString('hex');   // persist tx.id + hex BEFORE broadcast
```

14.4 Python [VERIFIED-DEVNET]

`examples/python/obs_tx.py` (pure Python + `ecdsa`) re-implements the address, encoding, gas, signing and txid logic. `python3 obs_tx.py --selftest` prints **OK** only if address, body, digest, txid and signed bytes equal the core's test vector. A transaction signed by this Python code was **accepted and included by a local devnet node** (`/tx/submit` 200, confirmed at height 27).

```
import obs_tx as o
amt = o.parse_obs("12.5")                                  # exact, no floats
_, q = o.rpc(NODE, "POST", "/wallet/quote", {"address": sender})
t = o.sign_payment(PRIVATE_KEY_HEX, "obs", 7777, "2.0.2", to_address, amt,
                   q["nextNonce"], q["protocolTime"] + 600, body_memo="wd-1001")
# persist t["txId"] and t["signedTxHex"] FIRST, then:
status, r = o.rpc(NODE, "POST", "/tx/submit", {"tx": t["signedTxHex"]})
```

14.5 CLI [VERIFIED-DEVNET] (commands run against devnet; mainnet needs `--network mainnet`)

```
node obsidian-core/dist/index.js wallet new --network mainnet     # offline wallet (prints the phrase once)
node obsidian-core/dist/index.js version                           # core, protocol, paramsHash
node obsidian-core/dist/index.js health --network mainnet          # queries the local node's /health   [VERIFIED-SRC]
node obsidian-core/dist/index.js doctor --network mainnet          # diagnostics
    ↪ [VERIFIED-SRC]
```

14.6 Signing test vector (devnet, public test keys — never use for value) [VERIFIED-DEVNET]

Generated by `examples/make-test-vector.mjs` from the real core; reproduced by Python (`--selftest`). Chain id 7780, prefix `dobs`, protocol `2.0.2`, private keys `0x…01` (sender) and `0x…02` (recipient).

Field	Value
sender	`dobs1w508d6qejxtdg4y5r3zarvary0c5xw7kdy6yr9`
recipient	`dobs1q6hag67dl53wl99vzg42z8eyzfz2xlkv86tz63`
amount / gas	12.5 OBS = `1250000000000000000` seals / `2500000000000000` seals
nonce / validUntil / body memo	0 / 1792524800 / `dep-1001`
signing digest	`8f97b1e27b0057a7001a7e18941b4f7ffeee2749ffcdd1131cb9eead09fc2ce2`
signature	`98acc5e87f72a42c329ab633bfc4bc03d68afb291befffc304107b2352c3a93621a9be91a82fe2661804f72b6a98c8466667512a6ce31fb8805b9447f5b75f0f`
txid	`39f59ace03dd680479e98052173e6944e2d01fe2bdcfc019e1f9e7bf87084ede`

Body hex (`string to ‖ u128 amount ‖ string memo`): `0000002b646f627331713668616736376465333337763939767a6734327a3865797a667a32786c6b763836747a3633000000000000000000ad78ebc5ac6200000000000086465702d31303031`. The full signed transaction hex is in `examples/test-vector.json`.

15. Test and acceptance checklist

The complete, fill-in version is `OBSIDIAN_EXCHANGE_TEST_CHECKLIST.md`. Class **T** = can be run on a testnet or devnet; class **M** = needs a controlled mainnet run with real funds. The Result column records what was **actually executed during preparation of this document, on a local devnet** (`verification/devnet-verify.mjs`: 59 automated checks + 3 after-restart checks; Python and Node examples run separately). It is **not** a result on testnet or mainnet: those are `NOT TESTED` until the exchange runs them.

Result	Count
PASS	24
FAIL	1
NOT TESTED	11
NOT IMPLEMENTED	3

ID	Requirement	Class	Result (devnet run)	Evidence
T-01	Node reachable; `/livez` 200	T	**PASS**	B (health reads)

ID	Requirement	Class	Result (devnet run)	Evidence
T-02	Readiness gate: `/readyz` 200 before integration use; 503 when syncing	T	**PASS**	B (readyz read; not-ready path from source)
T-03	Identity pin: chainId, networkId, genesisId, paramsHash equal the operator-published values	T	**PASS**	B1 (devnet identity); mainnet values NOT TESTED
T-04	Address generation offline; correct prefix; deterministic	T	**PASS**	A1, A2
T-05	Address validation: bad checksum, empty, wrong network, uppercase, mixed case	T	**PASS**	A3, E (invalid address ×3, uppercase, mixed)
T-06	Watch-only deposit addresses from an xpub equal phrase-derived addresses	T	**PASS**	examples/node/xpub-deposit-addresses.mjs selftest
T-07	Test deposit: send to a deposit address; detect it by block scan with unmasked recipient	T	**PASS**	D3–D10 (scan composition itself ILLUSTRATIVE)
T-08	Balance equals deposited amount; sender paid amount + gas	T	**PASS**	D11, D12
T-09	Transaction lookup: PENDING then confirmed with height, blockHash, amount	T	**PASS**	D5, D6
T-10	Confirmations increase with the tip	T	**PASS**	G2
T-11	Deterministic local signing; third-party implementation reproduces vector	T	**PASS**	Python `--selftest`; node harness
T-12	Broadcast; idempotent resubmit while pending; resubmit after inclusion not applied twice	T	**PASS**	D3, D4, F5
T-13	Fees: gas quote equals formula; min and max gas	T	**PASS**	D0, D12, E (wrong gas); max cap from source
T-14	Withdrawal to invalid / other-network address is refused	T	**PASS**	E (other-network prefix; invalid address)
T-15	Insufficient funds refused; quote `sufficient:false`	T	**PASS**	E (insufficient funds, quote)
T-16	Nonce rules: wrong nonce, nonce+1 while pending, same-nonce clash (409)	T	**PASS**	E (future nonce), F2, F3
T-17	Minimum transfer, expired validUntil, wrong chain id, oversize memo, malformed hex	T	**PASS**	E matrix
T-18	RPC failure handling: 4xx matrix, JSON-RPC errors are HTTP 200, 429	T	**PASS**	G3–G5, H1
T-19	RPC failure handling: timeouts, connection refused, failover to second node	T	**NOT TESTED**	exchange-side
T-20	Node restart: height continues, tx found, balance unchanged	T	**PASS**	`after-restart` phase (3 checks)
T-21	Reconciliation arithmetic per address (received − swept − gas = balance)	T	**PASS**	F4 (balance after sweep)
T-22	Full reconciliation job (ledger vs chain, hot wallet, stuck withdrawals)	T	**NOT TESTED**	exchange-side
T-23	Reorg handling: deposit marked REORGED, rescan, no double credit	T	**NOT TESTED**	needs a multi-producer fork; none in this run
T-24	Finality read on your own node (loopback) and hidden from proxied callers	T	**PASS**	B2–B4 (visibility); finality advancing NOT TESTED (needs ≥ 2 producers)
T-25	Finality advancing on a multi-producer network	T	**NOT TESTED**	not exercised
T-26	Memo behaviour: body vs envelope memo, retrieval	T	**PASS**	D6b, D13, D15
T-27	Rate limit: 429 `ERR_RATE_LIMITED`	T	**PASS**	H1
T-28	`Retry-After` header on 429	T	**FAIL**	H2: DOC-MISMATCH (header absent; documentation says present)

ID	Requirement	Class	Result (devnet run)	Evidence
T-29	No secrets in harness artifacts	T	**PASS**	grep of verification-results.json for privateKey/recoveryPhrase = 0
T-30	Private routes blocked at your proxy (`/metrics`, `/rpc`, `/peers`, `/finality`)	M	**NOT TESTED**	exchange-side; node has no check on `/metrics` (B6)
T-31	TLS on externally reachable proxy; certificate chain; HSTS	M	**NOT TESTED**	exchange-side
T-32	Controlled mainnet deposit: small real deposit detected, confirmed, credited by policy, swept	M	**NOT TESTED**	requires live mainnet and funds
T-33	Controlled mainnet withdrawal: small real withdrawal signed offline, broadcast, confirmed	M	**NOT TESTED**	requires live mainnet and funds
T-34	Mainnet identity: live genesis triple equals operator-published triple	M	**NOT TESTED**	live network unreachable here
T-35	Seed/peer connectivity and sync of an exchange node on mainnet	M	**NOT TESTED**	live network unreachable here
T-36	Key security review: custody, HSM, approvals, backups, restore drill	M	**NOT TESTED**	exchange-side
T-37	WebSocket / webhook delivery	M	**NOT IMPLEMENTED**	GAP-05
T-38	Paginated address history beyond 100 records	M	**NOT IMPLEMENTED**	GAP-04
T-39	Public finality endpoint	M	**NOT IMPLEMENTED**	GAP-06 (by design operator-local)

16. Gap analysis

Capabilities an exchange would normally expect and what the repository actually provides. Severity: **H** blocks or materially complicates a safe integration, **M** needs a workaround, **L** convenience/documentation. Evidence for each is a source file and, where marked, a devnet run.

ID	Sev	Capability
GAP-01	H	Unmasked transaction data on the public REST routes
GAP-02	H	Block transaction list lacks recipient and amount
GAP-03	M	Body memo is retrievable only from recent block events
GAP-04	M	History is capped and has no cursor
GAP-05	M	No push notifications (WebSocket, webhook, SSE)
GAP-06	H	Finality value is not available to public callers
GAP-07	M	No SDK and an inaccurate format document
GAP-08	H	No confirmed public mainnet RPC endpoint, seed list or join peer
GAP-09	M	One in-flight transaction per sending address
GAP-10	M	Wallet tooling for exchanges is minimal
GAP-11	H	Genesis identity of the live mainnet is not derivable from the repository
GAP-12	H	No circulating-supply definition or field
GAP-13	M	`/metrics` has no caller check and exposes the finalized height
GAP-14	L	`Retry-After` documented but not sent on 429
GAP-15	L	Documentation disagrees on the minimum producers for finality
GAP-16	M	Memo semantics are two-fold and unsearchable
GAP-17	L	No multi-recipient payment
GAP-18	M	Explorer and live endpoints not verifiable here
GAP-19	H	Upgrade and migration policy for a value-bearing mainnet

ID	Sev	Capability
GAP-20	M	No node-level authentication, quotas per client or allow-list
GAP-21	M	No explicit DROPPED/EXPIRED transaction status
GAP-22	M	Security-review evidence for listing due diligence

GAP-01 — Unmasked transaction data on the public REST routes (severity H)

Item	Value
Source files	obsidian-core/src/indexer/indexer.ts `serializeTransaction`, `maskAddress`; rpc/server.ts `transaction`, `addressHistory`
Why it matters	`GET /tx/{id}` and `/address/{addr}` return sender/recipient as `first10…last6`. A masked prefix/suffix is not a safe key for crediting a deposit.
Workaround	Use JSON-RPC `gettransaction {txId}` on your own node (unmasked recipient, exact amount): VERIFIED-DEVNET. Discover txids by scanning `/block/{h}`.
Recommended fix	Add an operator-only (or opt-in) unmasked `GET /tx/{id}?full=1`, or return the full recipient on `/block/{h}` transactions.
How to verify the fix	`GET /block/{h}` lists `recipient` and `amount` for PAYMENT; devnet-verify checks D8/D9 are updated.

GAP-02 — Block transaction list lacks recipient and amount (severity H)

Item	Value
Source files	rpc/server.ts `block` (fields: id, type, sender, nonce, gas, validUntil, size)
Why it matters	A scanner cannot filter a block for its own addresses without one extra call per PAYMENT.
Workaround	N+1 scan: `/block/{h}` → `gettransaction` for each PAYMENT (§9.2). Verified in parts on devnet.
Recommended fix	Include `recipient`, `amount`, `memo` for PAYMENT in the block's transaction summaries (or a dedicated `GET /block/{h}/payments`).
How to verify the fix	One `/block/{h}` call returns everything needed to detect a deposit.

GAP-03 — Body memo is retrievable only from recent block events (severity M)

Item	Value
Source files	rpc/server.ts `block` (`indexer.events(200).filter(height)`); executors/payment.ts (memo in body and event only)
Why it matters	The memo written by the project's wallets (body memo) is not in `/tx/{id}`. The event lookup searches the newest 200 events, so older blocks return none.
Workaround	Do not depend on memos. Use one address per customer. If you must read a body memo, do it within minutes of inclusion and persist it.
Recommended fix	Store the body memo in the index record and return it from `/tx/{id}` and `gettransaction`.
How to verify the fix	`GET /tx/{id}` returns `memo` for a body-memo payment (devnet check D6b flips).

GAP-04 — History is capped and has no cursor (severity M)

Item	Value
Source files	rpc/server.ts `addressHistory` (limit ≤ 100); indexer.ts `MAX_ADDRESS_HISTORY = 5000` (in-memory per address)
Why it matters	No offset/cursor; an active deposit address or a hot wallet loses old entries from the answer.
Workaround	Scan blocks and keep your own ledger; use `/address` as a cross-check only.
Recommended fix	Add `before`/`cursor` pagination bound to height/index.
How to verify the fix	Page through > 100 records of one address.

GAP-05 — No push notifications (WebSocket, webhook, SSE) (severity M)

Item	Value
Source files	obsidian-core/src/rpc/server.ts (HTTP only)
Why it matters	Latency and load: you must poll.
Workaround	Poll `/health` + `/blocks?from=` every 2–5 s from your own node (cheap).
Recommended fix	An opt-in event stream (`newBlock`, `newTransaction`) on the private RPC.
How to verify the fix	A subscriber receives a block event within one block time.

GAP-06 — Finality value is not available to public callers (severity H)

Item	Value
Source files	rpc/server.ts `mayShowFinality`, `finality`, `readyz`; docs: v2.0.3 release
Why it matters	Since 2.0.3 a caller that is not on the node's loopback (without proxy headers) receives no finalized height. Finality also needs ≥ 2 established producers.
Workaround	Run your own node (§12) and read `/finality` locally, or set `OBSIDIAN_RPC_PUBLIC_PEER_DETAIL=true` on a private node. Otherwise use depth thresholds (§5.6).
Recommended fix	Operator decision: publish a signed/attested finalized checkpoint through a dedicated, documented endpoint for integrators.
How to verify the fix	An integrator with an allow-listed key reads `finalizedHeight` from a public host. Devnet verified B2–B4 (visibility rule).

GAP-07 — No SDK and an inaccurate format document (severity M)

Item	Value
Source files	docs/transaction-format.md (says bech32m, nonce u64, type u8); transactions/encode.ts, crypto/bech32.ts, protocol/encoding.ts (bech32, u32 nonce, u16 type)
Why it matters	Teams that implement from the document will produce rejected transactions.
Workaround	Use §8.2 of this document, the test vector (§14.6) and the Python/Node references in `examples/`.
Recommended fix	Correct `docs/transaction-format.md`; publish a versioned signing spec + vectors; package the signer as a library.
How to verify the fix	A third-party implementation passes the vector (done here for Python).

GAP-08 — No confirmed public mainnet RPC endpoint, seed list or join peer (severity H)

Item	Value
Source files	obsidian-core/config/mainnet.json (`seedNodes: []`, RPC on 127.0.0.1); docs/trusted-domains.md (names only)
Why it matters	An exchange cannot connect without a seed/peer from the operator, and cannot verify any live hostname from the repository.
Workaround	Obtain from the operator: seed `host:port` list, one HTTPS node to `join` from, and the genesis triple; verify out of band.
Recommended fix	Publish an operator-signed endpoint/seed list and a status page.
How to verify the fix	`join` succeeds against the published peer and `/health` matches the published triple.

GAP-09 — One in-flight transaction per sending address (severity M)

Item	Value
Source files	rpc/server.ts `submitTransaction` (state-nonce simulation, `ERR_NONCE_PENDING`); blockchain/mempool.ts
Why it matters	A single hot address can send ≈ one transaction per block (5 s). VERIFIED-DEVNET: nonce+1 is refused while nonce is pending.
Workaround	A pool of hot sending addresses; queue per address.
Recommended fix	Accept consecutive future nonces into the mempool (bounded), or batch payments.

Item	Value
How to verify the fix	Submit nonce and nonce+1 back-to-back; both are accepted and included in order.

GAP-10 — Wallet tooling for exchanges is minimal (severity M)

Item	Value
Source files	obsidian-core/src/index.ts `wallet new` (index 0 only, phrase on stdout); crypto/mnemonic.ts (coin type 7777 is a placeholder, not a registered SLIP-44 value)
Why it matters	No CLI for HD ranges, xpub export, offline signing or hardware wallets; hardware-wallet/third-party-wallet support for coin type 7777 is unconfirmed.
Workaround	`examples/node/xpub-deposit-addresses.mjs` and `withdraw.mjs`; custom signer.
Recommended fix	Add `wallet derive/xpub/sign` CLI commands; register a SLIP-44 coin type; test with a hardware wallet.
How to verify the fix	CLI-only offline flow reproduces the test vector; hardware wallet signs a devnet tx.

GAP-11 — Genesis identity of the live mainnet is not derivable from the repository (severity H)

Item	Value
Source files	docs/mainnet-launch.md (key-less genesis id/hash; gate keys committed at `init`); genesis/*
Why it matters	VERIFIED-DEVNET: committing a mining-gate key changed the devnet genesis id. The documented mainnet id may therefore differ from the live one.
Workaround	Pin the operator-published triple (genesisId, genesisHash, paramsHash) and compare with your node.
Recommended fix	Publish the live genesis document and its hashes in the release notes.
How to verify the fix	`GET /genesis` on an independent node equals the published triple.

GAP-12 — No circulating-supply definition or field (severity H)

Item	Value
Source files	rpc/server.ts `supply` (totalSupply, maxSupply, poolBalance, marketEscrow, genesisIssued, minedSupply)
Why it matters	Listing platforms and data aggregators require a circulating-supply figure and definition.
Workaround	None that is authoritative. Operator must define it (what to exclude: Mining Pool, escrow, genesis holder, treasury).
Recommended fix	Define it in docs and add `circulatingSupplyObs` to `/supply`.
How to verify the fix	Value equals the published formula on an independent node.

GAP-13 — `/metrics` has no caller check and exposes the finalized height (severity M)

Item	Value
Source files	rpc/server.ts `metrics`; devnet check B6 (served to a proxied caller)
Why it matters	Exposing the RPC port, or proxying `/metrics`, leaks the value the project withholds from `/status`.
Workaround	Deny `/metrics` at the proxy; scrape locally.
Recommended fix	Apply `mayShowFinality`/operator-local rule to `/metrics`.
How to verify the fix	Proxied caller gets 404 on `/metrics`.

GAP-14 — `Retry-After` documented but not sent on 429 (severity L)

Item	Value
Source files	docs/api.md line ~113; rpc/server.ts (429 without the header)
Why it matters	Clients cannot honour a server-provided delay. VERIFIED-DEVNET: DOC-MISMATCH (header absent).
Workaround	Exponential backoff.
Recommended fix	Send `Retry-After`, or correct the doc.
How to verify the fix	429 response carries the header.

GAP-15 — Documentation disagrees on the minimum producers for finality (severity L)

Item	Value
Source files	docs/mainnet-launch.md (“at least three”); protocol/params.ts `finality.minProducers: 2`; docs/finality-v1.9.0.md (two); devnet message “at least 2”
Why it matters	Affects when finality can exist on mainnet.
Workaround	Treat the code (2) as authoritative; confirm the live producer count with the operator.
Recommended fix	Align the documents.
How to verify the fix	Docs and `/finality.minProducers` agree.

GAP-16 — Memo semantics are two-fold and unsearchable (severity M)

Item	Value
Source files	executors/payment.ts (body memo), transactions/encode.ts (envelope memo), indexer.ts (indexes only the envelope memo)
Why it matters	Wallets differ in which memo they set; no search by memo exists.
Workaround	One address per customer; never require a memo.
Recommended fix	Unify and index the body memo; add search.
How to verify the fix	A memo-tagged deposit is found by memo.

GAP-17 — No multi-recipient payment (severity L)

Item	Value
Source files	executors/payment.ts (single `to`)
Why it matters	Mass withdrawals cost one transaction (and one block slot per sender) each.
Workaround	Pool of sending addresses.
Recommended fix	Add a batch payment transaction (consensus change).
How to verify the fix	N recipients in one tx.

GAP-18 — Explorer and live endpoints not verifiable here (severity M)

Item	Value
Source files	docs/explorer.md; docs/trusted-domains.md
Why it matters	Listing files need a working explorer URL and live endpoints.
Workaround	Operator to supply and the exchange to test.
Recommended fix	Publish explorer URL(s) and monitor them.
How to verify the fix	Open a tx in the explorer by txid.

GAP-19 — Upgrade and migration policy for a value-bearing mainnet (severity H)

Item	Value
Source files	docs/mainnet-launch.md (“Protocol 2.0.2 activation is genesis-bound … no rolling upgrade or automatic state migration”); protocol/params.ts
Why it matters	A consensus-parameter change is a new protocol identity and genesis id. Exchanges need to know how balances and addresses carry over, and the notice period.
Workaround	Pin versions; treat any announcement as a halt-and-reconfirm event.
Recommended fix	Publish an upgrade policy and a migration design with notice periods.
How to verify the fix	Policy document exists and is signed by the operator.

GAP-20 — No node-level authentication, quotas per client or allow-list (severity M)

Item	Value
Source files	rpc/server.ts (`allowRequest` per-IP limiter; `rpcTrustProxy`)
Why it matters	A shared public node cannot serve exchanges with guaranteed capacity.
Workaround	Reverse proxy with auth/allow-lists, or your own node.
Recommended fix	Optional API keys and per-key limits at the node or a reference proxy configuration.
How to verify the fix	Two keys get independent quotas.

GAP-21 — No explicit DROPPED/EXPIRED transaction status (severity M)

Item	Value
Source files	rpc/server.ts `transaction` (PENDING | confirmed | 404)
Why it matters	404 is ambiguous: never seen, expired, dropped, or rolled back.
Workaround	Persist signed bytes and `validUntil`; decide 'dropped' after `validUntil`; rebroadcast the same bytes before that.
Recommended fix	Return `EXPIRED`/`DROPPED` for known txids.
How to verify the fix	Expired tx returns a status.

GAP-22 — Security-review evidence for listing due diligence (severity M)

Item	Value
Source files	(none in repository)
Why it matters	Exchanges ask for third-party review reports.
Workaround	Operator supplies any report that exists; the repository contains its own tests and a threat model (docs/threat-model.md).
Recommended fix	Provide the report(s).
How to verify the fix	Report supplied.

17. Appendices

Appendix A — Error code catalogue

Code	HTTP	Meaning
`ERR_MALFORMED`	400	body not a JSON object; txid/hex malformed; cannot decode; memo too long (inside ERR_REJECTED)
`ERR_BAD_ADDRESS`	400	address invalid for this network
`ERR_WRONG_CHAIN_ID`	400	transaction chain id differs from the node's
`ERR_REJECTED`	400	screening/simulation failed; message is `ERR_<CAUSE>: …` (BAD_NONCE, BAD_GAS, AMOUNT_ZERO, EXPIRED, NOT_YET_VALID, BAD_ADDRESS, INSUFFICIENT_FUNDS, BAD_SIGNATURE, MALFORMED, VERSION_MISMATCH, UNKNOWN_TX_TYPE …)
`ERR_FORBIDDEN`	403	origin not allowed, or submission disabled on this node
`ERR_NOT_FOUND`	404	unknown route, block, transaction, or `/finality` for a non-operator caller
`ERR_METHOD_NOT_ALLOWED`	405	wrong HTTP method (`allow` lists the right ones)
`ERR_NONCE_PENDING`	409	a different transaction with the same sender and nonce is pending (`pendingTxId`)
`ERR_BODY_TOO_LARGE`	413	request body over 512 KiB
`ERR_RATE_LIMITED`	429	per-IP rate limit, or mempool admission refused
`ERR_INTERNAL`	500	node bug
`-32000 / ERR_METHOD`	200	JSON-RPC: every method error, including unknown method, bad params, `submittransaction`

Appendix B — Where existing project documents disagree with the code

Document	Says	Code / observation
docs/transaction-format.md	says addresses are bech32m, nonce `u64`, type `u8`	code: bech32, `u32`, `u16` (`encode.ts`, `bech32.ts`, `encoding.ts`); the executed test vector confirms the code
docs/api.md (~line 113)	429 is sent with `Retry-After`	no header observed on devnet; `server.ts` sends none
docs/mainnet-launch.md	finality needs ≥ 3 established producers	`params.ts` `minProducers: 2`; `finality-v1.9.0.md` says two; devnet message says two
`/metrics` described as private		no caller check at the node (served to a proxied caller on devnet)
docs/mainnet-launch.md genesis id/hash	presented as 'a genuine mainnet node's' id	correct only for a genesis without committed mining-gate keys (GAP-11)

Appendix C — Evidence index: executed devnet checks

Harness `verification/devnet-verify.mjs` (62 checks recorded: 1 DOC-MISMATCH, 61 PASS). Environment: local devnet chain id 7780, `obsidian-core` 2.0.4, Node.js 22, one node, throw-away keys; output file `verification/verification-results.json` contains no private keys or phrases.

Check	Result	Detail
A1 wallet derives locally, address has the network prefix	PASS	43 chars
A2 same phrase + same path derives the same address (deterministic)	PASS	
A3 a mainnet-prefix address is not valid on devnet	PASS	
B1 node reports chain id and network id	PASS	
B2 /status loopback caller sees finality fields (operator-local)	PASS	
B3 /status with a Via header hides finality fields	PASS	

Check	Result	Detail
B4 /finality is 404 for a proxied caller	PASS	
B5 /peers is counts-only for a proxied caller (no addresses)	PASS	
B6 /metrics has NO caller check at the node (so the reverse proxy must block it)	PASS	served to a proxied caller
B7 /tx/encode returns the canonical PAYMENT body hex	PASS	
B8 /tx/encode decodes a PAYMENT body	PASS	
C0 hot wallet is eligible for the one-off devnet claim	PASS	{"eligible":true,"reason":"OK"}
C1 funding claim accepted	PASS	
C2 hot wallet funded	PASS	100000.000166666666666666 OBS
D0 /tx/gas equals expectedGas()	PASS	0.002500000000000000
D1 txid is known to the signer before broadcast	PASS	
D2 simulate says valid	PASS	
D3 submit accepted (mempool admission, NOT confirmation)	PASS	
D4 identical re-submit while pending is an idempotent 200 duplicate (or already mined)	PASS	status=200 duplicate=true
D5 pending lookup is PENDING or already confirmed	PASS	PENDING
D6 confirmed lookup carries height, blockHash, amount, confirmations	PASS	height=6 confirmations=1
D6b a PAYMENT BODY memo is NOT part of the /tx record (it is only in the PAYMENT event)	PASS	memo field absent from /tx
D7 REST /tx masks sender and recipient	PASS	
D8 JSON-RPC gettransaction returns the FULL recipient address	PASS	
D9 /block/<h> lists the tx with the full sender but no recipient or amount	PASS	
D10 /address/<deposit> lists the deposit (masked counterparties)	PASS	
D11 balance equals the deposited amount exactly (gas is paid by the sender on top)	PASS	12.500000000000000000
D12 sender paid amount + gas	PASS	
D13 /block/<h> events carry a PAYMENT event with the body memo (recent blocks only)	PASS	
D14 payment with an envelope memo accepted	PASS	
D15 an ENVELOPE memo is returned by /tx	PASS	
E wrong chain id (7777) is refused	PASS	status=400 code=ERR_WRONG_CHAIN_ID ERR_WRONG_CHAIN_ID: this node follows devnet (chain id 7780); the transaction targets
E wrong gas is refused	PASS	status=400 code=ERR_REJECTED ERR_BAD_GAS: gas mismatch: expected 200000000000000, received 200000000000001
E below minimum transfer is refused	PASS	status=400 code=ERR_REJECTED ERR_AMOUNT_ZERO: amount is below the protocol minimum of 1000000000000 seal(s)
E expired validUntil is refused	PASS	status=400 code=ERR_REJECTED ERR_EXPIRED: transaction expired at 1792524515 (protocol time 1792524575)

Check	Result	Detail
E wrong (future) nonce is refused	PASS	status=400 code=ERR_REJECTED ERR_BAD_NONCE: expected nonce 3, received 8
E malformed hex is refused	PASS	status=400 code=ERR_MALFORMED tx must be a hex-encoded signed transaction
E truncated transaction is refused	PASS	status=400 code=ERR_MALFORMED cannot decode transaction: canonical encoding: unexpected end of data
E memo longer than 256 bytes is refused	PASS	status=400 code=ERR_REJECTED ERR_MALFORMED: memo too long
E recipient with another network prefix is refused	PASS	status=400 code=ERR_REJECTED ERR_BAD_ADDRESS: recipient "obs1el3lj7e37gn2qjdp4nf5gdjdmfdtrygs2uvssu" is not a valid dobs
E insufficient funds is refused	PASS	status=400 code=ERR_REJECTED ERR_INSUFFICIENT_FUNDS: sender cannot fund the gas used for mempool priority
E quote reports sufficient=false for an unfunded address	PASS	
E invalid address "obs1invalid" → 400 ERR_BAD_ADDRESS	PASS	
E invalid address "" → 400 ERR_BAD_ADDRESS	PASS	
E invalid address "dobs1qqqqqqqqq" → 400 ERR_BAD_ADDRESS	PASS	
E all-uppercase address is canonicalised to lowercase in the response	PASS	
E mixed-case address is refused	PASS	
F1 sweep accepted	PASS	
F2 queued nonce+1 from the same sender is refused before the first is mined (documented behaviour)	PASS	status=400 code=ERR_REJECTED
F3 same sender+nonce clash is 409 ERR_NONCE_PENDING (or the first was already mined)	PASS	status=409 code=ERR_NONCE_PENDING
F4 after the sweep the deposit address holds only the rounding remainder	PASS	500000000000
F5 re-submitting a MINED tx is refused (nonce already used), not applied twice	PASS	status=400 ERR_BAD_NONCE: expected nonce 1, received 0
G1 /blocks?from= returns canonical entries with txCount	PASS	
G2 confirmations grow with the tip	PASS	1 → 3
G3 JSON-RPC errors are HTTP 200 with an error object	PASS	
G4 unknown txid is 404 ERR_NOT_FOUND	PASS	
G5 malformed txid is 400 ERR_MALFORMED	PASS	
H1 per-IP rate limit answers 429 ERR_RATE_LIMITED	PASS	after 217 requests
H2 429 carries a Retry-After header (docs/api.md says it does)	DOC-MISMATCH	Retry-After=absent
node restart: height did not go backwards	PASS	before=8 after=14
node restart: confirmed tx is still found by id	PASS	
node restart: balance unchanged	PASS	500000000000

Additional executed evidence (not in the JSON): `examples/python/obs_tx.py --selftest` **OK**; `examples/node/xpub-deposit-addresses.mjs selftest` **OK**; a Python-signed payment accepted and included by a local devnet node; `examples/node/withdraw.mjs` accepted and included on devnet, and aborted when the node's chain id differed from the expected one; `obsidian-core wallet new --network devnet` produced a 24-word wallet.

Appendix D — Version history

Version	Date	Notes
1.0.0	2026-10-20	First edition. Based on obsidian-core 2.0.4 / protocol 2.0.2 (commit c29811e). All executable claims verified on a local devnet; no live-network claims.

Appendix E — Contact and distribution

- Contact: emoluxlabs@gmail.com **[OPERATOR]** (placeholder; replace before distribution).
- Intended label: **Public Technical Documentation**. It contains no secrets: the only keys in the package are the public test keys `0x…01` and `0x…02` (devnet vectors). Hosts in examples are placeholders.
- Source commit described: `c29811e0b8f854c7212071c521974fd1249fbd16` (tag `v2.0.4`).
- Regenerate everything: `python3 docs/exchange/build/build_docs.py && /home/user/pdfenv/bin/python docs/exchange/build/build_pdf.py` (needs reportlab); re-run the evidence with `docs/exchange/verification/devnet-verify.mjs` against a disposable devnet.

Appendix F — Test and acceptance checklist (fill-in)

Companion to `OBSIDIAN_EXCHANGE_INTEGRATION.md` v1.0.0 (obsidian-core 2.0.4 / protocol 2.0.2).

How to use. Class **T** tests can run on a testnet/devnet (`tobs`/`dobs` addresses, chain 7778/7780). Class **M** tests need a controlled mainnet run with a small amount of real OBS and must be approved by your risk owner. Allowed results: `PASS`, `FAIL`, `NOT TESTED`, `NOT IMPLEMENTED`. **Never record PASS for a test that was not executed.**

The column Devnet run (preparation) is what was actually executed by the preparers on a local devnet (`verification/devnet-verify.mjs` and the example scripts). The columns Testnet run and Mainnet run are for the exchange and start as `NOT TESTED`.

ID	Requirement	Class	Procedure	Devnet run (preparation)	Evidence	Testnet run	Mainnet run
T-01	Node reachable; `/livez` 200	T	GET /livez	**PASS**	B (health reads)	NOT TESTED	NOT TESTED
T-02	Readiness gate: `/readyz` 200 before integration use; 503 when syncing	T	GET /readyz	**PASS**	B (readyz read; not-ready path from source)	NOT TESTED	NOT TESTED
T-03	Identity pin: chainId, networkId, genesisId, paramsHash equal the operator-published values	T	GET /health, /genesis	**PASS**	B1 (devnet identity); mainnet values NOT TESTED	NOT TESTED	NOT TESTED
T-04	Address generation offline; correct prefix; deterministic	T	wallet new / deriveWallet	**PASS**	A1, A2	NOT TESTED	NOT TESTED
T-05	Address validation: bad checksum, empty, wrong network, uppercase, mixed case	T	local + POST /wallet/balance	**PASS**	A3, E (invalid address ×3, uppercase, mixed)	NOT TESTED	NOT TESTED
T-06	Watch-only deposit addresses from an xpub equal phrase-derived addresses	T	local	**PASS**	examples/node/xpub-deposit-addresses.mjs selftest	NOT TESTED	NOT TESTED
T-07	Test deposit: send to a deposit address; detect it by block scan with unmasked recipient	T	scan §9.2	**PASS**	D3–D10 (scan composition itself ILLUSTRATIVE)	NOT TESTED	NOT TESTED
T-08	Balance equals deposited amount; sender paid amount + gas	T	POST /wallet/balance	**PASS**	D11, D12	NOT TESTED	NOT TESTED

ID	Requirement	Class	Procedure	Devnet run (preparation)	Evidence	Testnet run	Mainnet run
T-09	Transaction lookup: PENDING then confirmed with height, blockHash, amount	T	GET /tx/{id}	**PASS**	D5, D6	NOT TESTED	NOT TESTED
T-10	Confirmations increase with the tip	T	GET /tx/{id}	**PASS**	G2	NOT TESTED	NOT TESTED
T-11	Deterministic local signing; third-party implementation reproduces vector	T	obs_tx.py	**PASS**	Python `--selftest`; node harness	NOT TESTED	NOT TESTED
T-12	Broadcast; idempotent resubmit while pending; resubmit after inclusion not applied twice	T	POST /tx/submit	**PASS**	D3, D4, F5	NOT TESTED	NOT TESTED
T-13	Fees: gas quote equals formula; min and max gas	T	POST /tx/gas	**PASS**	D0, D12, E (wrong gas); max cap from source	NOT TESTED	NOT TESTED
T-14	Withdrawal to invalid / other-network address is refused	T	submit	**PASS**	E (other-network prefix; invalid address)	NOT TESTED	NOT TESTED
T-15	Insufficient funds refused; quote `sufficient:false`	T	quote + submit	**PASS**	E (insufficient funds, quote)	NOT TESTED	NOT TESTED
T-16	Nonce rules: wrong nonce, nonce+1 while pending, same-nonce clash (409)	T	submit	**PASS**	E (future nonce), F2, F3	NOT TESTED	NOT TESTED
T-17	Minimum transfer, expired validUntil, wrong chain id, oversize memo, malformed hex	T	submit	**PASS**	E matrix	NOT TESTED	NOT TESTED
T-18	RPC failure handling: 4xx matrix, JSON-RPC errors are HTTP 200, 429	T	various	**PASS**	G3–G5, H1	NOT TESTED	NOT TESTED
T-19	RPC failure handling: timeouts, connection refused, failover to second node	T	kill the node; verify retry/failover	**NOT TESTED**	exchange-side	NOT TESTED	NOT TESTED
T-20	Node restart: height continues, tx found, balance unchanged	T	restart	**PASS**	`after-restart` phase (3 checks)	NOT TESTED	NOT TESTED

ID	Requirement	Class	Procedure	Devnet run (preparation)	Evidence	Testnet run	Mainnet run
T-21	Reconciliation arithmetic per address (received − swept − gas = balance)	T	arithmetic only	**PASS**	F4 (balance after sweep)	NOT TESTED	NOT TESTED
T-22	Full reconciliation job (ledger vs chain, hot wallet, stuck withdrawals)	T	run job daily	**NOT TESTED**	exchange-side	NOT TESTED	NOT TESTED
T-23	Reorg handling: deposit marked REORGED, rescan, no double credit	T	testnet fork drill	**NOT TESTED**	needs a multi-producer fork; none in this run	NOT TESTED	NOT TESTED
T-24	Finality read on your own node (loopback) and hidden from proxied callers	T	GET /finality	**PASS**	B2–B4 (visibility); finality advancing NOT TESTED (needs ≥ 2 producers)	NOT TESTED	NOT TESTED
T-25	Finality advancing on a multi-producer network	T	testnet with ≥ 2 mature producers	**NOT TESTED**	not exercised	NOT TESTED	NOT TESTED
T-26	Memo behaviour: body vs envelope memo, retrieval	T	GET /tx, /block	**PASS**	D6b, D13, D15	NOT TESTED	NOT TESTED
T-27	Rate limit: 429 `ERR_RATE_LIMITED`	T	burst GETs	**PASS**	H1	NOT TESTED	NOT TESTED
T-28	`Retry-After` header on 429	T	observe 429 headers	**FAIL**	H2: DOC-MISMATCH (header absent; documentation says present)	NOT TESTED	NOT TESTED
T-29	No secrets in harness artifacts	T	grep	**PASS**	grep of verification-results.json for privateKey/recoveryPhrase = 0	NOT TESTED	NOT TESTED
T-30	Private routes blocked at your proxy (`/metrics`, `/rpc`, `/peers`, `/finality`)	M	curl through the proxy	**NOT TESTED**	exchange-side; node has no check on `/metrics` (B6)	NOT TESTED	NOT TESTED
T-31	TLS on externally reachable proxy; certificate chain; HSTS	M	openssl s_client / SSL Labs	**NOT TESTED**	exchange-side	NOT TESTED	NOT TESTED
T-32	Controlled mainnet deposit: small real deposit detected, confirmed, credited by policy, swept	M	§9	**NOT TESTED**	requires live mainnet and funds	NOT TESTED	NOT TESTED
T-33	Controlled mainnet withdrawal: small real withdrawal signed offline, broadcast, confirmed	M	§8	**NOT TESTED**	requires live mainnet and funds	NOT TESTED	NOT TESTED

ID	Requirement	Class	Procedure	Devnet run (preparation)	Evidence	Testnet run	Mainnet run
T-34	Mainnet identity: live genesis triple equals operator-published triple	M	/genesis	NOT TESTED	live network unreachable here	NOT TESTED	NOT TESTED
T-35	Seed/peer connectivity and sync of an exchange node on mainnet	M	join + start	NOT TESTED	live network unreachable here	NOT TESTED	NOT TESTED
T-36	Key security review: custody, HSM, approvals, backups, restore drill	M	§13	NOT TESTED	exchange-side	NOT TESTED	NOT TESTED
T-37	WebSocket / webhook delivery	M	—	NOT IMPLEMENTED	GAP-05	NOT IMPLEMENTED	NOT IMPLEMENTED
T-38	Paginated address history beyond 100 records	M	—	NOT IMPLEMENTED	GAP-04	NOT IMPLEMENTED	NOT IMPLEMENTED
T-39	Public finality endpoint	M	—	NOT IMPLEMENTED	GAP-06 (by design operator-local)	NOT IMPLEMENTED	NOT IMPLEMENTED

Results summary (preparation, devnet)

Result	Count
PASS	24
FAIL	1
NOT TESTED	11
NOT IMPLEMENTED	3

Sign-off

Role	Name	Date	Signature
Exchange technical lead			
Exchange security			
Project operator			

Notes

- T-28 is a documentation mismatch observed on devnet (429 without `Retry-After`), reported as FAIL against the documentation.
- T-23 and T-25 need a multi-producer network; the preparation environment had one producer.
- Everything marked M needs the live network, which was not reachable from the preparation environment.